CS 5489: Machine Learning Cheat Sheet

December 10, 2025

Supervised Learning

Naive Bayes

Generative model: model the prior and class-conditional distribution (each class has a particular distribution of features) by observation / assumption to use MLE on class conditional probability and Bayes decision rule (BDR) to get target probability:

$$p(y|x) = \frac{p(x,y)}{p(x)} = \frac{p(x|y)\cdot p(y)}{p(x)} = \frac{p(x|y)\cdot p(y)}{\sum_y p(x|y)} \tag{1}$$

Naive Bayes: assume features are conditionally independent, i.e. $p(\boldsymbol{x} = (x_1, x_2)|y) = p(x_1|y)p(x_2|y)$.

(class) **Model** $p(y)$. *Bernoulli distribution*: $p(y) = \pi^{\mathbb{1}(y=1)} \cdot (1-\pi)^{\mathbb{1}(y=0)}$.

MLE: $\pi^* = \arg\max \sum_{i=1}^{N} \log p(y_i) \Rightarrow \pi = \frac{\#(y=1)}{\#(y=0)+\#(y=1)}$.

(observation) **Model** $p(x|y)$. ① *Gaussian*: $p(x|y=c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{(-\frac{1}{2\sigma_c^2}(x-\mu_c)^2)}$.

MLE: $\mu_c^* = \frac{1}{N}\sum_{i=1}^{N} x_i, {\sigma_c^*}^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu_c^*)^2$. $(x_i|y=c)$

② *Poisson*: $p(x|y=c) = \frac{1}{x!}e^{-\mu_c}{\mu_c}^x, x \in \{0, 1, 2, \cdots\}, (\mu_c = \bar{x}|y=c)$.

MLE: $\mu_c^* = \frac{1}{N}\sum_{i=1}^{N} x_i$. $(x_i|y=c)$

Laplace smoothing (smoothed MLE): $\pi_j = \frac{N_j+\alpha}{N+2\alpha}$ *to prevent overfitting*.

Naive Bayes Example – SMS spam

Bag-of-Words (BoW) model: 1. build vocabulary $\mathcal{V}$. 2. text t to vector $x \in \mathbb{R}^V$ `sklearn.feature_extraction.text.`

① count occurrence to vectorize: `CountVectorizer(stop_words=xx, max_features=xx)`

② *Term-Frequency (TF)*: $x_j = \frac{w_j}{|D|} = \frac{\#\text{ word } j}{\#\text{ words in document}}$.

③ *TF Inverse Document Frequency (TF-IDF)*: $x_j = \frac{w_j}{|D|}\log\frac{N}{N_j}$

$IDF(j) = \log\frac{N}{N_j} = \frac{\#\text{ documents}}{\#\text{ documents with word } j}$. `TfidfVectorizer`

④ `HashingVectorizer`

NB Gaussian $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2_{\boldsymbol{\Sigma}}}$

MLE: $\boldsymbol{\mu}^* = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i, \boldsymbol{\Sigma}^* = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$.

NB Multinomial: $p(\boldsymbol{x}|y) = \frac{(\sum_j x_j)!}{\prod_j x_j!}\left(\prod_j \pi_{j,y}^{x_j}\right)$, where x_j is the frequency of word j $(\sum_j x_j = 1)$, $\pi_{j,y}$ is the probability of word w_j in class y $(\sum_{j=1}^{V}\pi_{j,y} = 1)$.

L_1 & L_2

L_1 regularization: 1. treat all errors equally 2. encourage more sparsity 3. decision is more likely to be aligned with the coordinate axis

L_1 error: robust to outlier, no preference to reduce the larger errors.

Generative → Discriminative

Generative: class-conditional distribution (CCD) $\xrightarrow{\text{Bayes rule}}$ classifier $p(y|x)$, can be used to small dataset.
Discriminative: classifier $p(y|x)$ directly

$$\frac{p(y=1|x)}{p(y=2|x)} > 1 \Leftrightarrow \log\frac{p(y=1|x)}{p(y=2|x)} \overset{\text{Bayes}}{=} \log\frac{p(x|y=1)p(y=1)}{p(x|y=2)p(y=2)} > 0 \tag{1}$$

sharing σ^2: $\log\frac{p(x|y=1)}{p(x|y=2)} = \log\frac{\prod_{i=1}^{D}\mathcal{N}(x_i|\mu_i,\sigma^2)}{\prod_{i=1}^{D}\mathcal{N}(x_i|\nu_i,\sigma^2)} = \frac{1}{2\sigma^2}\sum_{i=1}^{D}\left[2(\mu_i - \nu_i)x_i - \mu_i^2 + \nu_i^2\right]$

$$\frac{p(y=1|x)}{p(y=2|x)} > 1 \Leftrightarrow \sum_{i=1}^{D}\underbrace{\frac{1}{\sigma^2}(\mu_i - \nu_i)}_{w_i}x_i + \underbrace{\frac{1}{2\sigma^2}\sum_{i=1}^{D}(\nu_i^2 - \mu_i^2) + \log\frac{\pi_1}{\pi_2}}_{b} > 0 \tag{2}$$

Linear (Binary) Classifier

$f(x) = w^\top x + b$. $f(x) > 0 \to$ class $1(y=1)$, $f(x) < 0 \to$ class $2(y=-1)$

Logistic Regression (Binary) Classifier

$f(x) = w^\top x + b, \sigma(z) = \frac{1}{1+e^{-z}}, p(y|x) = \sigma(y \cdot f(x))$:

$$p(y=+1|x) = \sigma(f(x)), \quad p(y=-1|x) = 1 - \sigma(f(x)) = \sigma(-f(x)) \tag{3}$$

MLE: $(w^*, b^*) = \arg\max_{w,b}\sum_{i=1}^{N}\log p(y_i|x_i) = \arg\min_{w,b}\sum_{i=1}^{N}\log(1 + e^{-y_i f(x_i)})$

$z_i \triangleq y_i f(x_i) > 0 \Rightarrow$ classified correctly, $z_i < 0 \Rightarrow$ wrongly, $z_i = 0 \Rightarrow$ boundary
Logistic regression only forms a linear decision surface.

Logistic loss function: $L(z_i) = \log(1 + \exp(-z_i))$. convex → one optimum

Regularization: $(w^*, b^*) = \arg\max_{w,b}\left[\log p(w) + \sum_{i=1}^{N}\log p(y_i|x_i)\right]$.

① Gaussian: $p(w) = \mathcal{N}(0, \frac{C}{2}I) \Rightarrow \log p(w) = -\frac{1}{C}w^T w +$ constant

$\Rightarrow (w^*, b^*) = \arg\max_{w,b}\frac{1}{C}w^T w + \sum_{i=1}^{N}\log(1 + \exp(-y_i f(x_i))),: w^T w = \sum_{j=1}^{d} w_j^2$.

Large $C \to$ big $w \Leftrightarrow$ Small $C \to$ small w. Equal to L_1 regularization.

Multiclass logistic regression

$$p(y=c|x) = \text{softmax}(f(x)) = \frac{e^{f_c(x)}}{e^{f_1(x)} + \cdots + e^{f_K(x)}}, f_c(x) = w_c^T x. \tag{4}$$

MLE: ① likelihood: $p(y|x) = \prod_{j=1}^{K} p(y=j|x)^{y_j}$. ② log-likelihood: $\log p(y|x)$.
③ (min) cross-entropy loss: $-\log p(y|x) = -\sum_{j=1}^{K} y_j \log p(y=j|x)$

$$\max_{\{w_j\}}\sum_{i=1}^{N}\log p(y_i|x_i) = \max_{\{w_j\}}\sum_{i=1}^{N}\sum_{j=1}^{K} y_{ij}\log p(y=j|x_i) \tag{5}$$

where $y = [y_1, \cdots, y_K]$ is one-hot vector.

Support Vector Machine (SVM)

Margin distance: $\gamma = \min_i d_i = \min_i \frac{|f(x_i)|}{\|w\|} = \min_i \frac{|w^\top x_i + b|}{\|w\|}$
Support vector: points on $y = w^\top x + b$.
Normalization: $\frac{|aw^T x_i + ab|}{||aw||} = \frac{|w^T x_i + b|}{||w||} \Rightarrow$ just let $f(x_i) = w^T x_i + b = 1$.

Maximize margin: $(\hat{w}, b) = \arg\max_{w,b}\frac{1}{\|w\|}$ s.t. $\min_i |f(x_i)| = 1$

$\Leftrightarrow (\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}\|w\|^2 = \frac{1}{2}w^\top w$ s.t. $y_i f(x_i) \geqslant 1, \forall\, i$. (for binary class)
Lagrangian: $L(x, \lambda) = f(x) - \lambda g(x)$
$\Rightarrow$ *SVM dual*: $\min L(w, \alpha) = \frac{1}{2}w^T w - \sum_i \alpha_i[y_i(w^T x_i + b) - 1], w = \sum_{i=1}^{N}\alpha_i y_i x_i$

$$\arg\max_{\alpha}\sum_i \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j x_i^T x_j \text{ s. t.} \sum_{i=1}^{N}\alpha_i y_i = 0, \quad \alpha_i \geqslant 0 \tag{1}$$

non-separable data: $(\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}w^\top w$, s.t. $y_i f(x_i) \geqslant 1 - \xi_i, \xi_i \geqslant 0, \forall\, i$

soft-SVM: $(\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}w^\top w + \sum_{i=1}^{N} C\xi_i$, s.t. $y_i f(x_i) \geqslant 1 - \xi_i, \xi_i \geqslant 0, \forall\, i$

objective function: $\arg\min_{w,b}\frac{1}{C}w^T w + \underbrace{\sum_{i=1}^{N}\max(0, 1 - y_i f(x_i))}_{\text{hinge loss}}$

prediction: need training data to calculate similarity $f(x) = \sum_i \alpha_i y_i k(x_i, x) + b$.

Tricks

① Normalization for Logistic regression and SVM, due to their sensitivity to absolute value. ② Apply weights to loss of unbiased data. ③ Bigger weights to loss of more important class. ④ Change threshold to one class.

Kernel SVM

idea: learn linear classifier in high-dim space $\phi(x) \in \mathbb{R}^D$ rather than $x \in \mathbb{R}^d$
kernel function: $k(x_i, x_j) = \phi(x_i)^\top\phi(x_j)$, which is $x_i^\top x_j$ in *dual SVM*
example: ① polynomial kernel: $k(x, x') = (x^T x')^p = (\sum_{i=1}^{d} x_i x_i')^p$
② RBF(radial basis function): $k(x, x') = e^{-\gamma\|x-x'\|^2}$. $\gamma\uparrow$: smooth function.
③ Laplacian kernel: $k(x, x') = \exp(-\alpha||x - x'||)$

kernel SVM: $\hat{y} = \text{sign}(\sum_{i=1}^{N}\alpha_i y_i k(x_i, x_*) + b)$, where

$$\alpha = \arg\max_{\alpha}\sum_i \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j k(x_i, x_j) \text{ s. t.} \sum_{i=1}^{N}\alpha_i y_i = 0, \quad \alpha_i \geqslant 0 \tag{2}$$

Kernel matrix: $K = \left[k(x_i, x_j)\right]_{i,j}$, where $k(x_i, x_j) = \phi(x_i)^\top\phi(x_j)$.

K is a positive semi-definite matrix, i.e. $z^T K z \geq 0, \forall z$.
Kernel computation for high dimension is of high cost. Memory usage is based on the number of support vectors kept.

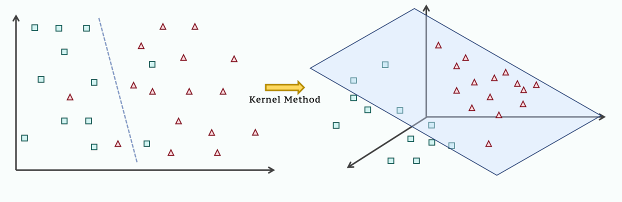


Figure 1: Kernel trick

小红书号:Ray0917

CS 5489: 机器学习备忘单

2025年12月10日

监督学习

朴素贝叶斯

生成式 模型：对 先验 和 类别 条件分布建模（每个
类别 通过观察具有特定的 特征 分布 / 假设为
使用MLE 关于类别条件概率和贝叶斯决策规则（BDR）来获得
目标概率：

$$p(y|x) = \frac{p(x,y)}{p(x)} = \frac{p(x|y)\cdot p(y)}{p(x)} = \frac{p(x|y)\cdot p(y)}{\sum_y p(x|y)} \tag{1}$$

朴素贝叶斯：假设 特征 是条件独立的, i.e. $p(\boldsymbol{x} = (x_1, x_2)|y) = p(x_1|y)p(x_2|y)$.

(类别) 模型 $p(y)$. 伯努利 分发： $p(y) = \pi^{\mathbb{1}(y=1)} \cdot (1-\pi)^{\mathbb{1}(y=0)}$.

MLE: $\pi^* = \arg\max \sum_{i=1}^{N} \log p(y_i) \Rightarrow \pi = \frac{\#(y=1)}{\#(y=0)+\#(y=1)}$.

（观察）模型 $p(x|y)$. ①高斯: $p(x|y=c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{(-\frac{1}{2\sigma_c^2}(x-\mu_c)^2)}$.

MLE: $\mu_c^* = \frac{1}{N}\sum_{i=1}^{N} x_i, {\sigma_c^*}^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu_c^*)^2$. $(x_i|y=c)$

②泊松: $p(x|y=c) = \frac{1}{x!}e^{-\mu_c}{\mu_c}^x, x \in \{0, 1, 2, \cdots\}, (\mu_c = \bar{x}|y=c)$.

MLE: $\mu_c^* = \frac{1}{N}\sum_{i=1}^{N} x_i$. $(x_i|y=c)$

拉普拉斯平滑（平滑的） **MLE):** $\pi_j = \frac{N_j+\alpha}{N+2\alpha}$ 防止过拟合。

朴素贝叶斯 示例 – 短信垃圾邮件

词袋 **(BoW)** 模型： 1. 建立词汇 $\mathcal{V}$. 2. 文本t 向量化 $x \in \mathbb{R}^V$ `sklearn.feature_extraction.text.`

①计数出现次数以向量化： `CountVectorizer(stop_words=xx, max_features=xx)`

② 词频 *(TF)*: $x_j = \frac{w_j}{|D|} = \frac{\#\text{ word } j}{\#\text{ words in document}}$.

③ *TF* 逆文档频率 *(TF-IDF)*: $x_j = \frac{w_j}{|D|}\log\frac{N}{N_j}$

$IDF(j) = \log\frac{N}{N_j} = \frac{\#\text{ documents}}{\#\text{ documents with word } j}$. `TfidfVectorizer`

④ `HashingVectorizer`

NB 高斯 $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2_{\boldsymbol{\Sigma}}}$

MLE: $\boldsymbol{\mu}^* = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i, \boldsymbol{\Sigma}^* = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$.

NB 多项式： $p(\boldsymbol{x}|y) = \frac{(\sum_j x_j)!}{\prod_j x_j!}\left(\prod_j \pi_{j,y}^{x_j}\right)$, 其中 x_j 是……的频率 词 j $(\sum_j x_j = 1)$, $\pi_{j,y}$ 是单词的概率 w_j 在 类别 y $(\sum_{j=1}^{V}\pi_{j,y} = 1)$.

L_1 & L_2

L_1 正则化： 1. 平等对待所有错误 2. 鼓励更多稀疏性 3.
决策更有可能与坐标轴对齐

L_1 错误：对异常值具有鲁棒性，对减小较大错误没有偏好。

生成式 → 判别式

生成式: 类别-条件分布 (CCD) $\xrightarrow{\text{贝叶斯定理}}$ 分类器 $p(y|x)$,
可以用于小型数据集。
判别式: 分类器 $p(y|x)$ 直接地

$$\frac{p(y=1|x)}{p(y=2|x)} > 1 \Leftrightarrow \log\frac{p(y=1|x)}{p(y=2|x)} \overset{\text{贝叶斯}}{=} \log\frac{p(x|y=1)p(y=1)}{p(x|y=2)p(y=2)} > 0 \tag{1}$$

共享 σ^2: $\log\frac{p(x|y=1)}{p(x|y=2)} = \log\frac{\prod_{i=1}^{D}\mathcal{N}(x_i|\mu_i,\sigma^2)}{\prod_{i=1}^{D}\mathcal{N}(x_i|\nu_i,\sigma^2)} = \frac{1}{2\sigma^2}\sum_{i=1}^{D}\left[2(\mu_i - \nu_i)x_i - \mu_i^2 + \nu_i^2\right]$

$$\frac{p(y=1|x)}{p(y=2|x)} > 1 \Leftrightarrow \sum_{i=1}^{D}\underbrace{\frac{1}{\sigma^2}(\mu_i - \nu_i)}_{w_i}x_i + \underbrace{\frac{1}{2\sigma^2}\sum_{i=1}^{D}(\nu_i^2 - \mu_i^2) + \log\frac{\pi_1}{\pi_2}}_{b} > 0 \tag{2}$$

线性（二进制） 分类器

$f(x) = w^\top x + b$. $f(x) > 0 \to$ 类别 $1(y=1)$, $f(x) < 0 \to$ 类别 $2(y=-1)$

逻辑回归（二元） 分类器

$f(x) = w^\top x + b, \sigma(z) = \frac{1}{1+e^{-z}}, p(y|x) = \sigma(y \cdot f(x))$:

$$p(y=+1|x) = \sigma(f(x)), \quad p(y=-1|x) = 1 - \sigma(f(x)) = \sigma(-f(x)) \tag{3}$$

MLE: $(w^*, b^*) = \arg\max_{w,b}\sum_{i=1}^{N}\log p(y_i|x_i) = \arg\min_{w,b}\sum_{i=1}^{N}\log(1 + e^{-y_i f(x_i)})$

$z_i \triangleq y_i f(x_i) > 0 \Rightarrow$ 分类正确, $z_i < 0 \Rightarrow$ 错误地 , $z_i = 0 \Rightarrow$ 边界
逻辑回归只形成线性决策面。

逻辑损失函数： $L(z_i) = \log(1 + \exp(-z_i))$. 凸函数 → 一个最优

正则化： $(w^*, b^*) = \arg\max_{w,b}\left[\log p(w) + \sum_{i=1}^{N}\log p(y_i|x_i)\right]$.

①高斯: $p(w) = \mathcal{N}(0, \frac{C}{2}I) \Rightarrow \log p(w) = -\frac{1}{C}w^T w +$ 常数

$\Rightarrow (w^*, b^*) = \arg\max_{w,b}\frac{1}{C}w^T w + \sum_{i=1}^{N}\log(1 + \exp(-y_i f(x_i))),: w^T w = \sum_{j=1}^{d} w_j^2$.

大 $C \to$ 大 $w \Leftrightarrow$ 小 $C \to$ 小 w. 等于 L_1 正则化。

多类逻辑回归

$$p(y=c|x) = \text{softmax}(f(x)) = \frac{e^{f_c(x)}}{e^{f_1(x)} + \cdots + e^{f_K(x)}}, f_c(x) = w_c^T x. \tag{4}$$

MLE: ①似然: $p(y|x) = \prod_{j=1}^{K} p(y=j|x)^{y_j}$. ②对数似然: $\log p(y|x)$.
③ (min) 交叉熵损失: $-\log p(y|x) = -\sum_{j=1}^{K} y_j \log p(y=j|x)$

$$\max_{\{w_j\}}\sum_{i=1}^{N}\log p(y_i|x_i) = \max_{\{w_j\}}\sum_{i=1}^{N}\sum_{j=1}^{K} y_{ij}\log p(y=j|x_i) \tag{5}$$

其中 $y = [y_1, \cdots, y_K]$ 是独热向量。

支持向量 机器 (SVM)

间隔距离： $\gamma = \min_i d_i = \min_i \frac{|f(x_i)|}{\|w\|} = \min_i \frac{|w^\top x_i + b|}{\|w\|}$
支持向量: 点数开启 $y = w^\top x + b$.
归一化： $\frac{|aw^T x_i + ab|}{||aw||} = \frac{|w^T x_i + b|}{||w||} \Rightarrow$ 就让 $f(x_i) = w^T x_i + b = 1$.

最大化 间隔： $(\hat{w}, b) = \arg\max_{w,b}\frac{1}{\|w\|}$ s.t. $\min_i |f(x_i)| = 1$

$\Leftrightarrow (\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}\|w\|^2 = \frac{1}{2}w^\top w$ s.t. $y_i f(x_i) \geqslant 1, \forall\, i$. （用于二进制 类别）
拉格朗日量： $L(x, \lambda) = f(x) - \lambda g(x)$
$\Rightarrow$ *SVM* 双重的: $\min L(w, \alpha) = \frac{1}{2}w^T w - \sum_i \alpha_i[y_i(w^T x_i + b) - 1], w = \sum_{i=1}^{N}\alpha_i y_i x_i$

$$\arg\max_{\alpha}\sum_i \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j x_i^T x_j \text{ s. t.} \sum_{i=1}^{N}\alpha_i y_i = 0, \quad \alpha_i \geqslant 0 \tag{1}$$

不可分数据： $(\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}w^\top w$, s.t. $y_i f(x_i) \geqslant 1 - \xi_i, \xi_i \geqslant 0, \forall\, i$

软-SVM： $(\hat{w}, b) = \arg\min_{w,b}\frac{1}{2}w^\top w + \sum_{i=1}^{N} C\xi_i$, s.t. $y_i f(x_i) \geqslant 1 - \xi_i, \xi_i \geqslant 0, \forall\, i$

目标函数： $\arg\min_{w,b}\frac{1}{C}w^T w + \underbrace{\sum_{i=1}^{N}\max(0, 1 - y_i f(x_i))}_{\text{合页损失}}$

预测：需要 训练 数据来计算相似性 $f(x) = \sum_i \alpha_i y_i k(x_i, x) + b$.

技巧

①逻辑回归的归一化和 SVM, 由于它们对⋯的敏感性
绝对值。②将权重应用于无偏数据的损失。③更大的权重到
失去更重要的类别。④将阈值更改为一个类别。

核函数 SVM

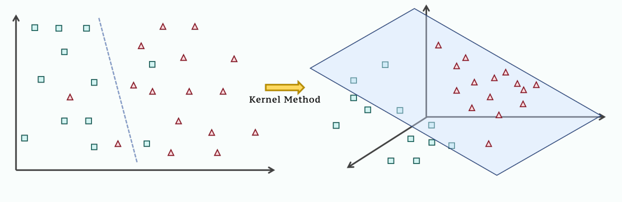
想法：在高维空间中学习线性 分类器 $\phi(x) \in \mathbb{R}^D$ 而不是 $x \in \mathbb{R}^d$
核函数： $k(x_i, x_j) = \phi(x_i)^\top\phi(x_j)$, 哪一个是$x_i^\top x_j$ 双重的 *SVM*
示例：①多项式 核函数： $k(x, x') = (x^T x')^p = (\sum_{i=1}^{d} x_i x_i')^p$
② RBF(径向基函数): $k(x, x') = e^{-\gamma\|x-x'\|^2}$. $\gamma\uparrow$: 光滑函数。
③拉普拉斯算子 核函数:$k(x, x') = \exp(-\alpha||x - x'||)$

核函数*SVM*: $\hat{y} = \text{sign}(\sum_{i=1}^{N}\alpha_i y_i k(x_i, x_*) + b)$, 其中

$$\alpha = \arg\max_{\alpha}\sum_i \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j k(x_i, x_j) \text{ s. t.} \sum_{i=1}^{N}\alpha_i y_i = 0, \quad \alpha_i \geqslant 0 \tag{2}$$

核函数 矩阵： $K = \left[k(x_i, x_j)\right]_{i,j}$, 其中 $k(x_i, x_j) = \phi(x_i)^\top\phi(x_j)$.

K 是一个半正定矩阵, i.e. $z^T K z \geq 0, \forall z$.
核函数 高维计算成本很高。内存使用情况是
根据保留的支持向量数量。



图形 1: 核函数 诡计

小红书号:Ray0917

Regression

Linear Regression. $\hat{y} = w_0 + w_1x_1 + \ldots + w_dx_d = \boldsymbol{w}^\top \boldsymbol{x} = f(\boldsymbol{x}_i)$, where $\boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top$, $\boldsymbol{x} = [1, x_1, \ldots, x_d]^\top$
Ordinary Least Squares (OLS). $\min_w \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2 = \min_w \|\boldsymbol{y} - \boldsymbol{X}^\top \boldsymbol{w}\|^2$, where $\boldsymbol{X} = [\boldsymbol{x}_1 \cdots \boldsymbol{x}_N]$ is the data matrix, $\boldsymbol{y} = [y_1, \ldots, y_N]$.
$\Rightarrow \min_w \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{y}^T\boldsymbol{X}^T\boldsymbol{w} + \boldsymbol{w}^T\boldsymbol{X}\boldsymbol{X}^T\boldsymbol{w} \Rightarrow \boldsymbol{w}^* = (\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{X}$, i.e. *pseudo-inverse*

Ridge Regression. $\min_w \alpha||\boldsymbol{w}||_2^2 + \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, where $||\boldsymbol{w}||_2^2 = \sum_{j=0}^d w_j^2$
$\Rightarrow \boldsymbol{w}^* = (\boldsymbol{X}\boldsymbol{X}^T + \alpha\boldsymbol{I})^{-1}\boldsymbol{X}\boldsymbol{y}$. (In OLS, $\boldsymbol{X}\boldsymbol{X}^\top$ may be non-invertible)

LASSO *(Least absolute shrinkage and selection operator).* $\min_w \alpha||\boldsymbol{w}||_1 + \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, where $||\boldsymbol{w}||_1 = \sum_{j=0}^d |w_j|$. *LASSO* encourage more sparsity

Sparsity Constraints. $\min_w \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, s.t. $||\boldsymbol{w}||_0 \leqslant K$, where $||\boldsymbol{w}||_0 =$ # non-zero entries. nonconvex and NP-hard v.s. *Ridge* and *LASSO* are convex

OMP *(Orthogonal Matching Pursuit).* Iteratively and greedily selects the most related feature to current residual error.
RANSAC *(**RAN**dom **SA**mple **C**onsensus).* Split the data into inliers (good data) and outliers (bad data) and learn the model only from the inliers.
Nonlinear Regression. $\hat{y} = \boldsymbol{w}^\top\phi(\boldsymbol{x})$, where $\boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top$.

Kernel Ridge Regression. $\min_w \|\boldsymbol{y} - \Phi\boldsymbol{w}\|_2^2 + \lambda\|\boldsymbol{w}\|_2^2$, where $\Phi = [\phi(\boldsymbol{x}_1), \ldots, \phi(\boldsymbol{x}_N)]^\top \in \mathbb{R}^{N\times d}, \boldsymbol{w} \in \mathbb{R}^d \Rightarrow \boldsymbol{w} = (\Phi^\top\Phi + \lambda\boldsymbol{I})^{-1}\Phi^\top\boldsymbol{y} \Rightarrow \hat{y} = \hat{\boldsymbol{k}}^\top(K + \lambda I)^{-1}\boldsymbol{y}$, where $K = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)] \in \mathbb{R}^{N\times N}$, $\hat{\boldsymbol{k}} = [k(\boldsymbol{x}_1, \hat{\boldsymbol{x}}), \ldots, k(\boldsymbol{x}_N, \hat{\boldsymbol{x}})]^\top \in \mathbb{R}^N$.

Gaussian Process Regression (GPR). *Gaussian process:* an sequential random variable set whose any finite subset is joint Gaussian distributed. $f(x) \sim \mathcal{GP}(m(x), k(x, x'))$ where f, m, k are value, mean and covariance function separately. $f_1, \ldots, f_N | x_1, \ldots, x_N \sim \mathcal{N}(0, K)$, where K is the kernel matrix.

GPR: ① observation noise: $y = f + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$, $p(y|f) = \mathcal{N}(y|f, \sigma^2 I)$ ② function prior: $f \sim \mathcal{GP}(0, k(x, x'))$ ③ train: $p(f|X, y) = \frac{p(y|X,f)p(f|X)}{p(y|X)} = \frac{p(y|f)p(f)}{p(y|X)}$ ④ inference: $p(\hat{f}|\hat{x}, X, y) = \int p(\hat{f}|\hat{x}, f)p(f|X, y)df$. ⑤ prediction: $p(f_*|x_*, \boldsymbol{X}, \boldsymbol{y}) = \mathcal{N}(f_*|\mu_*, \sigma_*^2)$, $\mu_* = \boldsymbol{k}_*^T(\boldsymbol{K} + \sigma^2 I)^{-1}\boldsymbol{y}$, $\sigma_*^2 = k_{**} - \boldsymbol{k}_*^T(\boldsymbol{K} + \sigma^2 I)^{-1}\boldsymbol{k}_*$, where $k_{**} = k(\boldsymbol{x}_*, \boldsymbol{x}_*)$.

Support Vector Regression (SVR). ① objective: $\min_{w,b} \sum_{i=1}^N \left|y_i - (\boldsymbol{w}^\top\boldsymbol{x}_i + b)\right|_\epsilon + \frac{1}{C}||\boldsymbol{w}||^2$, where $|z|_\epsilon = \begin{cases} 0, & |z| \leqslant \epsilon \\ |z| - \epsilon, & |z| > \epsilon \end{cases}$

Regression – Model Ensemble

Idea: combine multiple regression model together to form a better algorithm. ① ***bagging***: train multiple models from random selection of training data. ② ***boosting***: train multiple models which focus on errors made by previous one.

① ***Random Forest Regression.*** Use **bagging** to make an ensemble of *Decision Tree Regressor*. Random subset of data is used to train different tree, whose nodes use different features. The prediction is the average value of each tree. **RFR is sensitive to outliers.**
② ***XGBoost Regression.*** *(eXtreme **G**radient **B**oosting)* Use **boosting** to combine a set of less accurate models to create a accurate model. Weak learner fits the gradient of the loss: $h_t(\boldsymbol{x}) \approx \frac{dL}{df}$, $f_t(\boldsymbol{x}) = f_{t-1}(\boldsymbol{x}) - \alpha_t h_t(\boldsymbol{x}) \approx f_{t-1}(\boldsymbol{x}) - \alpha_t \frac{dL}{df_{t-1}}$.
Note: ① XGBoost / AdaBoost can run in parallel. ② not necessary to use different datasets to train *NN* when bagging.

Tricks

Improve accuracy: more features + complex features + complex model

Optimization

Adaptive learning rates: AdaGrad, RMSProp, Adam.

Unsupervised Learning

Dimensionality Reduction

Unsupervised learning. only considers the input data, with no output values, trying to discover inherent properties in the data.
Original data point $\boldsymbol{x} = \sum_{j=1}^p w_j\boldsymbol{v}_j \in \mathbb{R}^d$ can be represented as its corresponding weights: $\boldsymbol{w} = [w_1, \cdots, w_P] \in \mathbb{R}^p$.

Dimensionality Reduction. ① pre-process dataset for easier using ② make the results easier to understand ③ reduce computational cost and remove noise.
Goal: Transform high-dim vectors into low-dim vectors, where dimensions in the low-dim data ① represent co-occurring features in high-dim data ② may have semantic meaning.
Note: Not all dimensionality reduction algorithms are unsupervised.
Linear Dimensionality Reduction (LDR). Project the original data onto a lower-dim hyperplane and represent the data with coordinates in it.

LDR - Principal Component Analysis (PCA)

Principal Component Analysis (PCA). ① unsupervised method ② *goal:* preserve the variance of data as much as possible = minimize reconstruction error ③ choose basis vectors along the max variance (longest extent) of data, called principal components (PC).
Alg: ① subtract the mean of the data: $\bar{\boldsymbol{x}}_i = \boldsymbol{x}_i - \mu$ ② data covariance matrix: $\Sigma = \frac{1}{N}\sum_i \bar{\boldsymbol{x}}_i\bar{\boldsymbol{x}}_i^T = \frac{1}{N}\bar{\boldsymbol{X}}\bar{\boldsymbol{X}}^T$ ③ select top K eigenvectors of Σ as PC: $\boldsymbol{V} = [\boldsymbol{v}_1 \cdots \boldsymbol{v}_K]$ ④ project the data onto the PC: $\boldsymbol{w}_i = \boldsymbol{V}^T\boldsymbol{x}_i \in \mathbb{R}^K \to \hat{\boldsymbol{x}}_i = \boldsymbol{V}\boldsymbol{w}_i$.
Explained variance. each PC explains a percentage of the original data called explained variance.
Number of PC: till ① sum(explained variance) $\geqslant$ 95% ② performance is good.
Lack. preserving the variance sometimes won't help for classification: ① PCA doesn't consider which class the data belongs to ② When the "classification" signal is less than the "noise", PCA will make classification more difficult.

Random Projections. ① For k-components PCA in $\mathbb{R}^d$, complexity is $O(dk^2)$ ② generate random basis vectors sampled from a Gaussian ③ reduce computation at the expense of losing some accuracy (adding noise).
Johnson-Lindenstrauss lemma. carefully selecting the distribution of the random projection matrices will preserve the pairwise distances between any two samples of the dataset, within some error epsilon: $(1 - \epsilon)\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 < \|\boldsymbol{w}_i - \boldsymbol{w}_j\|^2 < (1 + \epsilon)\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2$.

Fisher's Linear Discriminant (FLD) / Linear Discriminant Analysis (LDA). ① supervised method ② *goal:* find a lower-dim space to maximize the class separation: 1. maximize the distance between projected means 2. minimize the projected variances ③ *model:* data from each class is modeled as a Gaussian.

Latent Semantic Analysis (LSA). ① document vector $\boldsymbol{x}_i$ is a BoW representation ② approximate $\boldsymbol{x}$ as a weighted sum of topic vectors $\hat{\boldsymbol{x}} = \sum_{n=1}^p w_p\boldsymbol{v}_p$ ③ *goal:* $\min_{v,w} \sum_i ||\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i||^2$ ④ *Advantage:* distances / similarities compare **topics** rather than words, a higher-level semantic representation. ⑤ *Lack:* frequency of a word can be negative in topic + weights for each topic can be negative.

Non-negative Matrix Factorization (NMF). ① $\min_{v\geqslant 0, w\geqslant 0} \sum_j ||\boldsymbol{x}_j - \hat{\boldsymbol{x}}_j||^2$ ② *sparse representation:* most topic weights for a document are zero. ③ not a probabilistic model and difficult to interpret.
Note: 1. linear Kernel *PCA* can reduce memory and computation ($\to O(d^2k)$) 2. *PCA* and *LSA* have closed-form solutions.

Non-Linear Dimensionality Reduction

Linear DR: model the data as "living" on a linear manifold (PCA, NMF, LSA)

Kernel PCA. ① run PCA on high-dim feature transformation $\phi(\boldsymbol{x}_i)$ ② *principal component:* $\boldsymbol{v} = \sum_{i=1}^n a_i\phi(\boldsymbol{x}_i)$ where a_i are learned weights ③ coefficient of new point $\boldsymbol{x}_*$ for $\boldsymbol{v}$: $w = \phi(\boldsymbol{x}_*)^T\boldsymbol{v} = \sum_{i=1}^n a_i\phi(\boldsymbol{x}_*)^T\phi(\boldsymbol{x}_i) = \sum_{i=1}^n a_ik(\boldsymbol{x}_*, \boldsymbol{x}_i) = \boldsymbol{k}_*^T\boldsymbol{a}$ ④ *learn weights:* center kernel $\tilde{\boldsymbol{K}} = (\boldsymbol{I} - \frac{1}{N}\boldsymbol{1}\boldsymbol{1}^T)\boldsymbol{K}(\boldsymbol{I} - \frac{1}{N}\boldsymbol{1}\boldsymbol{1}^T)$ where $\boldsymbol{K} = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{ij} \to$ select top-K eigenvalues and scale $\frac{1}{\sqrt{\lambda_j}}\boldsymbol{a}_j \to$ project $w_j = \boldsymbol{k}_*^T\boldsymbol{a}_j$.

Clustering – Parametric clustering

Data is set of vectors $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n\}$, where each data point is a vector $\boldsymbol{x} \in \mathbb{R}^d$.
Clustering. ① *Goal:* group similar data together ② *Cluster:* groups are called **clusters**, each data point is assigned with a cluster index ($\{1, \cdots, K\}$).

K-Means. ① *Idea:* each cluster is represented by a cluster center $\boldsymbol{c}_j \in \mathbb{R}^d$, $j \in \{1, \cdots, K\} \to$ assign each data point to the closest center: $(\boldsymbol{x}_i, z_i)$, $z_i \in \{1, \cdots, K\}$. ② *Objective:* minimize the total sum-squared difference between points and their centers: $\min \sum_{i=1}^n ||\boldsymbol{x}_i - \boldsymbol{c}_{z_i}||^2$. ③ *Method 1:* assume the assignments z_i are known ($C_j = \{\boldsymbol{x}_i | z_i = j\}$) $\to$ pick the cluster centers $\boldsymbol{c}_j = \operatorname{argmin} \sum i \in C_j ||\boldsymbol{x}_i - \boldsymbol{c}_j||^2 = \frac{1}{|C_j|}\sum \boldsymbol{x}_i \in C_j\boldsymbol{x}_i$. ④ *Method 2:* assume the clusters $\{\boldsymbol{c}_1, \cdots, \boldsymbol{c}_K\}$ are known $\to$ pick the assignments $z_i = \arg\min_{j\in\{1,\cdots K\}} ||\boldsymbol{x}_i - \boldsymbol{c}_j||^2$. ⑤ *Note:* result depends on initialization $\to$ random initializations & k-means++. ⑥ *Problem:* assume each cluster has a circular shape (Euclidean distance $\|\cdot\|$), thus can't handle skewed (elliptical) clusters. ⑦ *Note:* always converges.

Bag-of-X Representation. ① *Goal:* create a new quantized representation for samples. ② get k cluster center in many examples (vocabulary) $\to$ assign each part in samples a center $\to$ count the number of each center $\to$ k-dim vector.

Gaussian mixture model (GMM). ① use multivariate Gaussian model a cluster with an elliptical shape, where covariance matrix controls elliptical shape and mean controls location. ② *GMM:* weighted sum of Gaussian $p(\boldsymbol{x}) = \sum_{j=1}^K \pi_jN(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$, where π_j reflects how likely is the j-th cluster / Gaussian. ③ *MLE:* $\max_{\pi,\mu,\Sigma} \sum_{i=1}^N \log\sum_{j=1}^K \pi_jN(\boldsymbol{x}_i|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$. $\leftarrow$ hard to optimize $\to$ *EM* alg. ④ *Note:* use Mahalanobis distance; handle ellipse clusters

Expectation-Maximization (EM) algorithm. ① *Goal:* find the *MLE* solution $\hat{\theta} = \operatorname{argmax}_\theta \log p_\theta(\boldsymbol{x})$ when there are hidden variables z, where $p_\theta(\boldsymbol{x}) = \sum_z p(\boldsymbol{x}|z)p(z)$. ② *Solution:* iterate between estimating the hidden variables z and maximizing w.r.t the parameters θ. 1. *initialize* $\to$ 2. *E-step*: estimate the hidden variables as their expected value $\mathcal{Q}(\theta) \leftarrow \mathbb{E}_{z|\boldsymbol{x},\hat{\theta}}[\log p_\theta(\boldsymbol{x}, z)] \to$ 3. *M-step:* $\hat{\theta} \leftarrow \operatorname{argmax}_\theta \mathcal{Q}(\theta) \to$ 4. repeat until convergence.

EM alg. for GMM. ① *E-step:* soft assignment of point i to cluster j: $\hat{z}_{ij} = p(z_i = j|\boldsymbol{x}_i) = \frac{\pi_j\mathcal{N}(\boldsymbol{x}_i|\mu_j,\Sigma_j)}{\sum_k \pi_k\mathcal{N}(\boldsymbol{x}_i|\mu_k,\Sigma_k)}$ ② *M-step:* soft count $N_j = \sum_{i=1}^N \hat{z}_{ij} \to$ weight $\pi_j = N_j/N \to$ mean $\boldsymbol{\mu}_j = \frac{1}{N_j}\sum_{i=1}^N \hat{z}_{ij}\boldsymbol{x}_i \to$ variance $\Sigma_j = \frac{1}{N_j}\sum_{i=1}^N \hat{z}_{ij}(\boldsymbol{x}_i - \boldsymbol{\mu}_j)^2$.

Bayesian GMM. ① *Goal:* automatically selects K ② use Dirichlet distribution to model prior $p(\pi)$, mathematically $\pi_j = \frac{N_j+\alpha}{N+K\alpha}$ (similar to Laplace smoothing).
Clustering using KDE. ① automatically select cluster centers ② *Mode:* local maximum of probability density $\boldsymbol{\mu} = \operatorname{argmax}_x p(\boldsymbol{x})$, considered as centers. ③ *Find:* $\hat{\boldsymbol{\mu}} \leftarrow \hat{\boldsymbol{\mu}} + \eta\frac{d}{dx}p(\boldsymbol{x})$, convergence guaranteed by "Mean-shift algorithm".

Comparison of GMM and K-means. ① *Similarity:* 1. use iterative algorithms where first step assigns points, and the second step recalculates the cluster means 2. assume compact clusters (elliptical or circular shaped) 3. sensitive to initialization 4. can't decide clusters number K by itself ② *Difference:* 1. K-means uses hard-assignment (0/1) while *GMM* uses soft-assignments ($0 \sim 1$) 2. K-means assumes circular shape while *GMM* elliptical 3. *GMM* factor in the cluster size but K-means not.

回归

线性回归。 $\hat{y} = w_0 + w_1x_1 + \ldots + w_dx_d = \boldsymbol{w}^\top\boldsymbol{x} = f(\boldsymbol{x}_i)$, 其中 $\boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top$, $\boldsymbol{x} = [1, x_1, \ldots, x_d]^\top$
普通最小二乘 *(OLS).* $\min_w \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2 = \min_w \|\boldsymbol{y} - \boldsymbol{X}^\top\boldsymbol{w}\|^2$,
其中 $\boldsymbol{X} = [\boldsymbol{x}_1 \cdots \boldsymbol{x}_N]$ 是数据矩阵, $\boldsymbol{y} = [y_1, \ldots, y_N]$.
$\Rightarrow \min_w \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{y}^T\boldsymbol{X}^T\boldsymbol{w} + \boldsymbol{w}^T\boldsymbol{X}\boldsymbol{X}^T\boldsymbol{w} \Rightarrow \boldsymbol{w}^* = (\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{X}$, i.e. 伪逆

岭回归。 $\min_w \alpha||\boldsymbol{w}||_2^2 + \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, 其中 $||\boldsymbol{w}||_2^2 = \sum_{j=0}^d w_j^2$
$\Rightarrow \boldsymbol{w}^* = (\boldsymbol{X}\boldsymbol{X}^T + \alpha\boldsymbol{I})^{-1}\boldsymbol{X}\boldsymbol{y}$. （在OLS, $\boldsymbol{X}\boldsymbol{X}^\top$ 可能是不可逆的)

LASSO (最小绝对收缩和选择算子) $\min_w \alpha||\boldsymbol{w}||_1 + \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, 其中 $||\boldsymbol{w}||_1 = \sum_{j=0}^d |w_j|$. *LASSO* 鼓励更多稀疏性

稀疏性约束 $\min_w \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i))^2$, s.t. $||\boldsymbol{w}||_0 \leqslant K$, 其中 $||\boldsymbol{w}||_0 =$
非零项。非凸且NP难 v.s. 脊和 *LASSO* 是 凸函数

OMP （正交匹配追踪）。迭代且贪心地选择最
与当前残差相关的 特征。
RANSAC (随机抽样一致性)。将数据分为内点（良好数据）
以及异常值（错误数据），并仅从内点中学习模型。
非线性回归。 $\hat{y} = \boldsymbol{w}^\top\phi(\boldsymbol{x})$, 其中 $\boldsymbol{w} = [w_0, w_1, \ldots, w_d]^\top$.

核函数 岭回归。 $\min_w \|\boldsymbol{y} - \Phi\boldsymbol{w}\|_2^2 + \lambda\|\boldsymbol{w}\|_2^2$, 其中 $\Phi = [\phi(\boldsymbol{x}_1), \ldots, \phi(\boldsymbol{x}_N)]^\top \in \mathbb{R}^{N\times d}, \boldsymbol{w} \in \mathbb{R}^d \Rightarrow \boldsymbol{w} = (\Phi^\top\Phi + \lambda\boldsymbol{I})^{-1}\Phi^\top\boldsymbol{y} \Rightarrow \hat{y} = \hat{\boldsymbol{k}}^\top(K + \lambda I)^{-1}\boldsymbol{y}$, 其中 $K = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)] \in \mathbb{R}^{N\times N}$, $\hat{\boldsymbol{k}} = [k(\boldsymbol{x}_1, \hat{\boldsymbol{x}}), \ldots, k(\boldsymbol{x}_N, \hat{\boldsymbol{x}})]^\top \in \mathbb{R}^N$.

高斯 过程回归 ***(GPR).*** 高斯 过程：一个顺序随机
变量集，其任意有限子集都是联合 高斯 分布的。 $f(x) \sim \mathcal{GP}(m(x), k(x, x'))$ 其中 f, m, k 是值，均值 和协方差函数
分别地。$f_1, \ldots, f_N | x_1, \ldots, x_N \sim \mathcal{N}(0, K)$, 其中 K 是 核函数 矩阵。

GPR: ①观测噪声: $y = f + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$, $p(y|f) = \mathcal{N}(y|f, \sigma^2 I)$ ②函数 先验: $f \sim \mathcal{GP}(0, k(x, x'))$ ③训练: $p(f|X, y) = \frac{p(y|X,f)p(f|X)}{p(y|X)} = \frac{p(y|f)p(f)}{p(y|X)}$ ④ 推理: $p(\hat{f}|\hat{x}, X, y) = \int p(\hat{f}|\hat{x}, f)p(f|X, y)df$. ⑤预测: $p(f_*|x_*, \boldsymbol{X}, \boldsymbol{y}) = \mathcal{N}(f_*|\mu_*, \sigma_*^2)$, $\mu_* = \boldsymbol{k}_*^T(\boldsymbol{K} + \sigma^2 I)^{-1}\boldsymbol{y}$, $\sigma_*^2 = k_{**} - \boldsymbol{k}_*^T(\boldsymbol{K} + \sigma^2 I)^{-1}\boldsymbol{k}_*$, 其中 $k_{**} = k(\boldsymbol{x}_*, \boldsymbol{x}_*)$.

支持向量 回归 ***(SVR).*** ①目标: $\min_{w,b} \sum_{i=1}^N \left|y_i - (\boldsymbol{w}^\top\boldsymbol{x}_i + b)\right|_\epsilon + \frac{1}{C}||\boldsymbol{w}||^2$, 其中 $|z|_\epsilon = \begin{cases} 0, & |z| \leqslant \epsilon \\ |z| - \epsilon, & |z| > \epsilon \end{cases}$

回归 – 模型集成

想法：将多个回归模型结合在一起，形成一个更好的算法。
①装袋：训练 从随机选择的 训练 数据中使用多个模型。 ②
提升：训练 多个模型，重点关注前一个模型所犯的错误。

①随机森林回归。使用装袋法来构建决策树的集成
树回归器。数据的随机子集被用来 训练 不同的树，其
节点使用不同的特征。预测值是每棵树的平均值。
RFR 对离群点敏感。
②XGBoost 回归。 *(e**X**treme* 梯度提升) 使用提升来组合
一组较不精确的模型来创建一个精确的模型。弱学习器拟合
损失的梯度: $h_t(\boldsymbol{x}) \approx \frac{dL}{df}$, $f_t(\boldsymbol{x}) = f_{t-1}(\boldsymbol{x}) - \alpha_t h_t(\boldsymbol{x}) \approx f_{t-1}(\boldsymbol{x}) - \alpha_t \frac{dL}{df_{t-1}}$.
注意：①XGBoost/ AdaBoost 可以并行运行。②不必使用
不同的数据集用于 训练*NN* 在装袋时。

技巧

改进 准确率：更多 特征 + 复杂 特征 + 复杂模型

优化

自适应学习率： AdaGrad, RMSProp, Adam。

无监督学习

降维

无监督学习。只考虑输入数据，没有输出值，
试图发掘数据中的固有属性。
原始数据点 $\boldsymbol{x} = \sum_{j=1}^p w_j\boldsymbol{v}_j \in \mathbb{R}^d$ 可以表示为其相应的
权重: $\boldsymbol{w} = [w_1, \cdots, w_P] \in \mathbb{R}^p$.

降维。①预处理数据集以便更容易使用 ②使
结果更易于理解 ③降低计算成本并去除噪声。
目标：将高维向量转换为低维向量，其中的维度在
低维数据 ①表示高维数据 ②中共现的 特征 可能
具有语义意义。
注意：并非所有的降维算法都是无监督的。
线性降维 ***(LDR).*** 将原始数据投影到一个
低维超平面，并用其中的坐标来表示数据。

LDR - 主成分 分析 (PCA)

主成分 分析 ***(PCA).*** ①无监督方法 ②目标:
尽可能保留数据的方差 = 最小化重建误差
③沿着选择基向量 max 方差（数据的最长范围），称为
主成分 (PC).
算法：①减去数据的均值: $\bar{\boldsymbol{x}}_i = \boldsymbol{x}_i - \mu$ ②数据协方差矩阵:
$\Sigma = \frac{1}{N}\sum_i \bar{\boldsymbol{x}}_i\bar{\boldsymbol{x}}_i^T = \frac{1}{N}\bar{\boldsymbol{X}}\bar{\boldsymbol{X}}^T$ ③选择顶部 K 特征向量 的 Σ 作为PC: $\boldsymbol{V} = [\boldsymbol{v}_1 \cdots \boldsymbol{v}_K]$
④将数据投影到 PC: $\boldsymbol{w}_i = \boldsymbol{V}^T\boldsymbol{x}_i \in \mathbb{R}^K \to \hat{\boldsymbol{x}}_i = \boldsymbol{V}\boldsymbol{w}_i$.
解释了 方差。 每个 PC 解释称为原始数据百分比的内容
解释了 方差。
数量 *PC:* 直到① 总和(解释了 方差) $\geqslant$ 95% ②性能很好。
缺乏。保存 方差 有时对分类没有帮助: ① PCA
不考虑数据属于哪个 类别 ② 当“分类”
信号小于噪声 PCA 将使分类更加困难。

随机投影。①用于 k-成分 PCA 在$\mathbb{R}^d$, 复杂性是 $O(dk^2)$ ②
生成从 高斯 采样的随机基向量 ③ 减少计算
以牺牲一些准确率（增加噪声）为代价。
Johnson-Lindenstrauss 引理 仔细选择……的分布
随机投影矩阵将保留任意点之间的成对距离
数据集的两个样本，在某个误差 ε 范围内: $(1 - \epsilon)\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 < \|\boldsymbol{w}_i - \boldsymbol{w}_j\|^2 < (1 + \epsilon)\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2$.

费舍尔线性判别 ***(FLD) /*** 线性判别分析 ***(LDA).*** ①
监督方法 ②目标：找到一个低维空间以最大化 类别
分离： 1. 最大化投影均值之间的距离 2. 最小化
预计方差 ③模型：来自每个 类别 的数据被建模为 高斯。

潜在语义分析 ***(LSA).*** ①文档向量 $\boldsymbol{x}_i$ 是 BoW 表示
②近似 $\boldsymbol{x}$ 作为主题向量的加权和 $\hat{\boldsymbol{x}} = \sum_{n=1}^p w_p\boldsymbol{v}_p$ ③
目标: $\min_{v,w} \sum_i ||\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i||^2$ ④优势：距离 / 比较主题的相似点
而不是文字，而是更高层次的语义表示。⑤缺失：频率
一个词的含义在话题上可能是消极的+ 权重 对于每个主题可能都是负面的。

非负矩阵分解 ***(NMF).*** ① $\min_{v\geqslant 0, w\geqslant 0} \sum_j ||\boldsymbol{x}_j - \hat{\boldsymbol{x}}_j||^2$ ②
稀疏表示：大多数文档的主题 权重 为零。③不是一个
概率模型且难以解释。
注意: 1. 线性 核函数 *PCA* 可以减少内存和计算 ($\to O(d^2k)$)
2. *PCA* 且 *LSA* 有封闭形式的解。

非线性降维

线性 *DR:* 将数据建模为“存在于”线性流形上 (PCA, NMF, LSA)
核函数 ***PCA.*** ①跑 PCA 在高维 特征 转换 $\phi(\boldsymbol{x}_i)$ ②主
成分: $\boldsymbol{v} = \sum_{i=1}^n a_i\phi(\boldsymbol{x}_i)$ 其中 a_i 被学习的 权重 ③新的系数
点 $\boldsymbol{x}_*$ 对 $\boldsymbol{v}$: $w = \phi(\boldsymbol{x}_*)^T\boldsymbol{v} = \sum_{i=1}^n a_i\phi(\boldsymbol{x}_*)^T\phi(\boldsymbol{x}_i) = \sum_{i=1}^n a_ik(\boldsymbol{x}_*, \boldsymbol{x}_i) = \boldsymbol{k}_*^T\boldsymbol{a}$
④学习 权重：中心 核函数 $\tilde{\boldsymbol{K}} = (\boldsymbol{I} - \frac{1}{N}\boldsymbol{1}\boldsymbol{1}^T)\boldsymbol{K}(\boldsymbol{I} - \frac{1}{N}\boldsymbol{1}\boldsymbol{1}^T)$ 其中 $\boldsymbol{K} = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{ij} \to$ 选择前K 特征值 并缩放 $\frac{1}{\sqrt{\lambda_j}}\boldsymbol{a}_j \to$ 项目 $w_j = \boldsymbol{k}_*^T\boldsymbol{a}_j$.

聚类 – 参数聚类

数据是一组向量 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n\}$, 每个数据点都是一个向量 $\boldsymbol{x} \in \mathbb{R}^d$.
聚类。①目标：将相似的数据分组 ②簇：分组称为
簇，每个数据点都被分配了一个簇索引 ($\{1, \cdots, K\}$).

K-均值。①概念：每个簇由一个簇中心表示 $\boldsymbol{c}_j \in \mathbb{R}^d$, $j \in \{1, \cdots, K\} \to$ 将每个数据点分配到最近的中心: $(\boldsymbol{x}_i, z_i)$, $z_i \in \{1, \cdots, K\}$. ②目标：最小化总平方差
点及其中心: $\min \sum_{i=1}^n ||\boldsymbol{x}_i - \boldsymbol{c}_{z_i}||^2$. ③方法 *1:* 假设这个
分配 z_i 被知道 ($C_j = \{\boldsymbol{x}_i | z_i = j\}$) $\to$ 选择 簇 个中心 $\boldsymbol{c}_j = \operatorname{argmin} \sum i \in C_j ||\boldsymbol{x}_i - \boldsymbol{c}_j||^2 = \frac{1}{|C_j|}\sum \boldsymbol{x}_i \in C_j\boldsymbol{x}_i$. ④方法 *2:* 假设 簇
$\{\boldsymbol{c}_1, \cdots, \boldsymbol{c}_K\}$ 被知道 $\to$ 选择 分配 $z_i = \arg\min_{j\in\{1,\cdots K\}} ||\boldsymbol{x}_i - \boldsymbol{c}_j||^2$. ⑤
注意：结果取决于初始化 $\to$ 随机初始化 & k-均值 ++.
⑥问题：假设每个 簇 都是圆形（欧几里得距离 $\|\cdot\|$),
因此无法处理偏斜（椭圆形）簇。⑦ 注意：总是会收敛。

X的袋表示。①目标：为…创建一种新的量化表示
样本。②获取 k 簇 在许多例子中居中（词汇） $\to$ 分配每个
样本中的一部分中心 $\to$ 计算每个中心的数量 $\to$ k维向量。

高斯 混合模型 ***(GMM).*** ①使用多变量 高斯 模型 a 簇
具有椭圆形状，其中协方差矩阵控制椭圆形状
并且 均值 控制位置。 ② *GMM:* 高斯的加权和 $p(\boldsymbol{x}) = \sum_{j=1}^K \pi_jN(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$, 其中 π_j 反映第 j 个 簇 的可能性有多大 / 高斯。
③ *MLE:* $\max_{\pi,\mu,\Sigma} \sum_{i=1}^N \log\sum_{j=1}^K \pi_jN(\boldsymbol{x}_i|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$. $\leftarrow$ 难以优化 $\to$ *EM* 算法
④注意：使用马氏距离；处理椭圆 簇

期望最大化 ***(EM)*** 算法。①目标：找到 *MLE* 解决方案
$\hat{\theta} = \operatorname{argmax}_\theta \log p_\theta(\boldsymbol{x})$ 当存在隐藏变量时 z, 其中 $p_\theta(\boldsymbol{x}) = \sum_z p(\boldsymbol{x}|z)p(z)$. ②解决方案：在估计隐藏变量之间进行迭代 z
并最大化 w.r.t 参数 θ. 1. 初始化 $\to$ 2. E步：估计
隐藏变量作为它们的期望值 $\mathcal{Q}(\theta) \leftarrow \mathbb{E}_{z|\boldsymbol{x},\hat{\theta}}[\log p_\theta(\boldsymbol{x}, z)] \to$ 3. *M-*
步骤: $\hat{\theta} \leftarrow \operatorname{argmax}_\theta \mathcal{Q}(\theta) \to$ 4. 重复直到收敛。

EM 算法用于***GMM.*** ①E步：点的软分配 i 到 簇 j: $\hat{z}_{ij} = p(z_i = j|\boldsymbol{x}_i) = \frac{\pi_j\mathcal{N}(\boldsymbol{x}_i|\mu_j,\Sigma_j)}{\sum_k \pi_k\mathcal{N}(\boldsymbol{x}_i|\mu_k,\Sigma_k)}$ ②M步：软计数 $N_j = \sum_{i=1}^N \hat{z}_{ij} \to$ 权重 $\pi_j = N_j/N$
$\to$ 均值 $\boldsymbol{\mu}_j = \frac{1}{N_j}\sum_{i=1}^N \hat{z}_{ij}\boldsymbol{x}_i \to$ 方差 $\Sigma_j = \frac{1}{N_j}\sum_{i=1}^N \hat{z}_{ij}(\boldsymbol{x}_i - \boldsymbol{\mu}_j)^2$.

贝叶斯 ***GMM.*** ①目标：自动选择 K ②使用狄利克雷分布来
模型 先验 $p(\pi)$, 数学上 $\pi_j = \frac{N_j+\alpha}{N+K\alpha}$ (类似于 拉普拉斯平滑).
使用聚类 ***KDE.*** ①自动选择 簇 中心 ②模式：本地
概率密度的最大值 $\boldsymbol{\mu} = \operatorname{argmax}_x p(\boldsymbol{x})$, 被认为是中心。 ③
查找: $\hat{\boldsymbol{\mu}} \leftarrow \hat{\boldsymbol{\mu}} + \eta\frac{d}{dx}p(\boldsymbol{x})$, 由均值-移位算法保证的收敛性。

比较 ***GMM*** 和 K-均值。①相似性: 1. 使用迭代算法
第一步分配点数，第二步重新计算 簇
均值 2. 假设紧凑型簇（椭圆形或圆形） 3. 敏感的
初始化 4. 无法决定簇数字 K 本身 ②差异: 1. K-
均值 使用硬-分配 (0/1) 当 *GMM* 使用软-分配 ($0 \sim 1$)
2. K-均值 假设圆形 *GMM* 椭圆的 3. *GMM* 考虑到
簇 尺寸但 K-均值 不是。

Non-parametric clustering – Mean-shift algorithm

①*Idea:* iteratively shift towards the largest concentration of points ② *Alg.* start from an initial point $\boldsymbol{x}$ → find the neighbors to $\boldsymbol{x}$ within some radius (bandwidth) → set $\boldsymbol{x}$ as the mean of the neighbors points → repeat till convergence. ***Getting Clusters.*** ① Run for many initial points $\{\boldsymbol{x}_i\}$ → converged points are centers, points that converge to the same center belong to the same cluster. ② *Number of clusters:* implicitly controlled by the bandwidth, where larger bandwidth creates less clusters.

Non-parametric clustering – Spectral Clustering

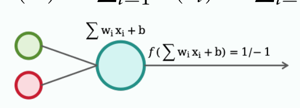
Non-compact clusters. K-means, GMM, and Mean-Shift assume that all clusters are compact, i.e. circles or ellipses, thus can't handle non-compact clusters. ***Spectral Clustering.*** ① *Idea:* 1. estimate clusters using pair-wise *affinity* between points 2. clustering with a graph formulation 3. cut edges that are small ② *Affinity / Similarity:* use kernel function $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$, like RBF kernel ③ *Graph formulation:* node is data point, edge weight is affinity $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$. ④ *Sensitivity to gamma:* small γ in RBF → far away points are still considered similar. ⑤ *Advantage*: can handle non-compact shape clusters / on a manifold which can form a connected graph.

Name	Cluster Shape	Principle	Advantages	Disadvantages
K-Means	circular	minimize distance to center	scalable (MiniBatchKMeans)	sensitive to initialization; local minima. must choose K.
GMM	elliptical	maximum likelihood	supports elliptical shapes	sensitive to initialization; local minima. must choose K.
Bayesian GMM	elliptical	maximum marginal likelihood	automatically selects K via concentration parameter	slow; sensitive to initialization.
Mean-Shift	compact	move toward local mean	automatically selects K via bandwidth	can be slow.
Spectral Clustering	irregular	graph-based	handles any connected cluster shape	must choose K; cannot assign new points; kernel matrix expensive.

Deep Learning

Multi-layer Perceptron

Perceptron. Model a single neuron $y = f(w^T x) = f(\sum_{j=0}^{d} w_j x_j) = \begin{cases} 1 & \text{if } w^\top x \geq 0 \\ -1 & \text{otherwise} \end{cases}$, *loss*: $E(w) = \sum_{i=1}^{N} L(z_i) = \sum_{i=1}^{N} \max(0, -z_i), z_i = y_i w^\top x_i$.



Perceptron Alg. / SGD. $w \leftarrow w + \eta y_i x_i = w - \eta g_w(x_i)$, where x_i is misclassified *Result (1st in ML)*: 1. only converge on linearly separable data; 2. # iteration $\sim \frac{1}{m}$, where m is the separation (margin) between classes.
Multi-layer Perceptron now called *neural network*.

Multi-class Logistic regression. class labels $y \in \{1, \cdots, C\}$, class vector $\boldsymbol{y} \in \mathbb{R}^C$ is one-hot vector. Output $p(y = j|\boldsymbol{x}) = f_j(\boldsymbol{x}) = s_j(\boldsymbol{g}(\boldsymbol{x})) = \frac{\exp(g_j(\boldsymbol{x}))}{\sum_{k=1}^{C} \exp(g_k(\boldsymbol{x}))}$, $g_j(\boldsymbol{x}) = \boldsymbol{w}_j^T \boldsymbol{x}$, for $j \in \{1, \cdots, C\}$.
MLE: $\log p(\boldsymbol{y}|\boldsymbol{x}) = \log \prod_{j=1}^{C} f_j(\boldsymbol{x})^{y_j} = \sum_{j=1}^{C} y_j \log f_j(\boldsymbol{x}) = \boldsymbol{y}^T \log \boldsymbol{f}(\boldsymbol{x})$
$\boldsymbol{W}^* = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \log p(\boldsymbol{y}_i|\boldsymbol{x}_i) = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \boldsymbol{y}_i^T \log \boldsymbol{f}(\boldsymbol{x}_i) = \arg\min_{\boldsymbol{W}} \sum_{i=1}^{N} \{-\sum_{j=1}^{C} y_{ij} \log f_j(\boldsymbol{x}_i)\}$.
cross-entropy loss: $L(\boldsymbol{y}, \boldsymbol{f}) = -\sum_{j=1}^{C} y_j \log f_j(\boldsymbol{x})$.
chain rule: $\frac{dL}{d\boldsymbol{w}_j} = \frac{dL}{d\boldsymbol{g}} \frac{d\boldsymbol{g}^T}{d\boldsymbol{w}_j} = \frac{dL}{d\boldsymbol{f}} \frac{d\boldsymbol{f}}{d\boldsymbol{g}} \frac{d\boldsymbol{g}^T}{d\boldsymbol{w}_j} = \boldsymbol{x}(f_j(\boldsymbol{x}) - y_j)$.

Multi-layer Perceptron(MLP). Add hidden layers between inputs and outputs. $\boldsymbol{h} = f(\boldsymbol{W}^T \boldsymbol{x})$, where $f(\cdot)$ is the *activation function*.

Activation function: ① Sigmoid: $\mathbb{R} \mapsto [0, 1], f(x) = \frac{1}{1+e^{-x}}$ ② Tanh: $\mathbb{R} \mapsto [-1, 1], f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ③ Rectifier Linear Unit (ReLU): $f(x) = \max(0, x)$. ④ *Note:* ReLU yield sparse representations and train fast.

MLP - Part 2

Overfitting: the training loss decreases, but the validation loss increases.
Early stopping: stop training when the validation loss is stable (change below a threshold) for a number of iterations to prevent overfitting.

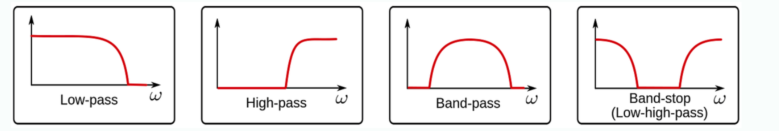
Universal Approximation Theorem: A *MLP* with one hidden layer and finite nodes can approximate any continuous function up to a desired error.
Vanishing Gradient problem: ① *Meaning:* Back-prop recursively multiplies gradients causing numerical values get smaller. ② *Reason:* 1. successive multiplications of small gradients + summation of large numbers of gradient paths for deeper network 2. activation functions that saturate at some points.

Convolutional Neural Networks (CNNs)

***Idea*:** Signal's features may have **locality** and **translation**, which can not be dealt with by *MLP*. To deal with it, use **local feature extractor** in the signal.

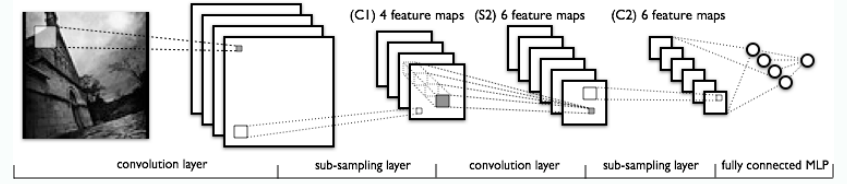
***1-D discrete convolution*:** $s(t) = x * w = \sum_a x(a)w(t - a)$.
***2-D discrete convolution*:** $s(n, m) = x * w = \sum_a \sum_b x(a, b)w(n - a, m - b)$.
***Property*:** $s = x * w \Rightarrow s(t - a) = x(t - a) * w(t)$. *Note:* convolution operation is equivalent to multiplication in the frequency domain.

***Interpretation 1*:** w is a filter on the frequency spectrum of signal x, e.g. *low-pass, high-pass, band-pass, low-high-pass, moving-average*.

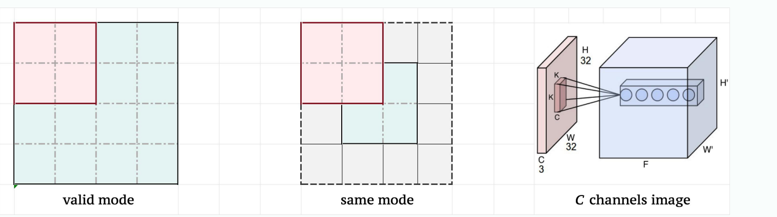


① transform to frequency domain $X(w)$ by *DFT* $X(\omega) = \sum_t x(t)e^{-i\omega t}$ where $e^{-i\omega t} = \cos(\omega t) - i\sin(\omega t)$ ② Do filtering by w ③ Back to time domain $x(t) = \frac{1}{2\pi} \int X(\omega)e^{i\omega t}d\omega$. Then $s(t) = x(t) * w(t) \Longleftrightarrow S(\omega) = X(\omega)W(\omega)$.
***Interpretation 2*:** w is a pattern (template) and try to find this pattern i.e. pattern w matches the local $x \Rightarrow x = \frac{w}{||w||}$.

***CNNs*:** series of convolutional layers, sub-sampling layers, and MLP classifier.



2D convolution filter: $h = f(\sum_{x,y} W_{x,y} P_{x,y})$, where W: template, P: part of image
***Convolution modes*:** ① *valid mode*: all $P_{x,y}$ has valid image values → shrinks by $(k - 1)/2$ ② *same mode*: zero-pad the border of the image → same size as the input image.



C channels image: $C \times H \times W \xrightarrow[C \times F \times K \times K]{F \text{ convolution filter}} F$ channels feature: $F \times H' \times W'$
***Spatial sub-sampling*:** ① *stride* $(s) \rightarrow (m - k_x + 1 - s) \times (n - k_y + 1 - s)$ ② *max-pooling* $\rightarrow W_{x,y}P_{x,y} = \max(P_{x,y})$, introduce translation invariance (robust) ③ *Effect:* reduce spatial resolution of feature maps thus faster inference speed.
***Receptive field*:** what pixels in the input affect a particular node $\rightarrow k_{i-1} + k_i - 1$
Advantages: ① extract the same features throughout the image ② pooling is robust to changes ③ less parameters than *FCNN*: $C \times H \times W \rightarrow F \times H \times W$, *FCNN* is $(CHW + 1) \times (FHW)$ and *CNN* is $(CKK + 1) \times F$

Regularization Methods

L2 regularization*:** $\hat{L} = L + \lambda\frac{1}{2}||\boldsymbol{w}||^2$. *SGD*: $\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \eta\frac{d\hat{L}}{d\boldsymbol{w}_k} = \boldsymbol{w}_k - \eta(\frac{dL}{d\boldsymbol{w}_k} + \lambda\boldsymbol{w}_k) = (1 - \eta\lambda)\boldsymbol{w}_k - \eta\frac{dL}{d\boldsymbol{w}_k}$, which is also called ***weight-decay.
*Note:*① *L2 regularization should be applied to all layers.* Because when applying L2 regularization on only one layer, the other layer can be adjusted to get the same network: $f(\boldsymbol{x}) = \boldsymbol{A}^T r(\boldsymbol{B}^T\boldsymbol{x}) = \boldsymbol{A}^T r(\frac{\epsilon}{\epsilon}\boldsymbol{B}^T\boldsymbol{x}) = \frac{1}{\epsilon}\boldsymbol{A}^T r(\epsilon\boldsymbol{B}^T\boldsymbol{x})$. ② equivalent to assuming a Gaussian prior on the weights and using MAP estimation.

***Early Stopping*:** limit the number of training iterations (how far $\boldsymbol{w}$ can move)
***Bagging*:** combine predictions from several models (model averaging).
Combine K models whose error is ϵ_i and $\text{MSE}_i = \mathbb{E}[\epsilon_i^2] = V$, then $\text{MSE}_c = \frac{V}{K} + \frac{K-1}{K}C$, where $C = \mathbb{E}[\epsilon_i\epsilon_j]$ is the correlation between errors of the i-th and j-th models. So MSE decreases when $C = 0$.

For *NN*, can use same dataset, but different initializations / hyper-parameters.
***Dropout*:** ① *Goal*: approximate a large ensemble of models by randomly removing an input/hidden node with some probability. ② *Training*: (1) "drop out" each node with probability p (2) train a reduced network in each iteration ③ *Testing:* use all the nodes for prediction and scale the output by $1 - p$ ④ *Effect*: (1) block some paths when computing gradients → so reduce vanishing gradient problem (2) reduces prediction error (3) reduces the effective capacity of the network (4) increases classifier robustness (not just rely on one feature).

***Data augmentation*:** artificially permute the data, using translation, flip, rotation, deformation, add pixel noise, scale, color, brightness, contrast, etc. to make 1. the dataset larger / model sees more variations of the data, 2. the network invariant to the permutations.

How to Go Deeper?

How many samples do we need? Suppose $\mathcal{N}$ is a feed-forward network with W weights, L layers, with a fixed piecewise activation function. Then $VCdim(\mathcal{N}) = O(WL\log W + WL^2)$. (Bartlett, Maiorov, Meir, 1998)
VC dimension is a statistical measure of function class capacity, and sample size needs to scale proportionally to. With same number of parameters, the deeper network requires more data (WL^2).

Rectified Linear Unit. $\text{ReLU}(z) = \max(0, z)$. *Advantage:* ① Easy and fast to train. ② Sparse representation, i.e. most nodes will output zero, reduce the vanishing gradient problem.

Batch Normalization. ① *Why?* Distribution of activations changes in 1 layer, then all layers are affected and need to change. ② *Batch-norm*: For each node in each layer, normalize outputs to $\hat{y}_i = \frac{y_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ over each mini-batch → scale-shift to $z_i = \gamma\hat{y}_i + \beta$, where γ, β are learnable. ③ *Effects*: reduce the effect of covariate shift → accelerate training + stable gradients + better generalization (no need for dropout / L_2-reg). ④ *Note:* better to put after a linear layer.

Learning Rate Schedule. ① *Linear change*: $\eta_k = (1 - \frac{k}{T})\eta_0 + \frac{k}{T}\eta_T$. ② *Decay*: $\eta_k = \frac{1}{1+\delta k}\eta_0$, where $0 < \delta < 1$. ③ *Adaptive schedule*: reduce the *lr* when the validation loss no longer improves.

Momentum. keep a running average of the gradients across mini-batches: $\boldsymbol{v}^{(t)} = \alpha\boldsymbol{v}^{(t-1)} - \eta\frac{dL}{d\boldsymbol{w}}, \boldsymbol{w}^{(t)} \leftarrow \boldsymbol{w}^{(t-1)} + \boldsymbol{v}^{(t)}$.

Transformer Learning & Fine-tuning

Transformer Learning. Rather than retrain the whole pre-trainied network, ① fix the lower layers that extract features ② train the last few layers to perform the new task. `*list(model.children())[:-1]`

非参数聚类 – 均值-移位算法

①想法：迭代地向点的最大集中区域移动 ②算法：开始从初始点 x → 寻找邻居到 x 在某个半径范围内 （带宽） → 设置x 正如邻居的均值所指出的 → 重复直到收敛。获取 簇。① 从多个初始点运行 $\{x_i\}$ → 收敛点是中心，汇聚到同一中心的点属于相同的 簇。 ② 簇 的数量：由 带宽 隐式控制，其中更大带宽 产生更少的 簇。

非参数聚类 – 谱聚类

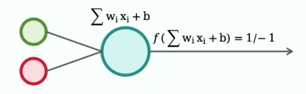
非紧凑型 簇。K-均值， GMM, 并且 均值-Shift 假设所有簇 是紧凑型的， i.e. 圆或椭圆，因此无法处理非紧凑 簇。谱聚类。① 思想： 1. 使用成对亲和力估计 簇在点之间 2. 使用图形方法进行聚类 3. 切掉小边 ②亲和力 / 相似度：使用 核函数 $k(x_i, x_j)$, 像 RBF 核函数 ③图公式：节点是数据点，边权重是亲和力 $k(x_i, x_j)$. ④敏感性到伽玛：小 γ 在 RBF → 遥远的点仍然被认为是相似的。 ⑤ 优势：可以处理非紧凑形状 簇 / 在一个流形上可以形成一个连通图。

名称	簇形状	原理	优点	缺点
K-均值	圆形的	将距离中心最小化	可扩展的（MiniBatchKMeans）	对初始化敏感；局部最小值。必须选择 K。
GMM	椭圆的	最大 似然	支持椭圆形	对初始化敏感；局部最小值。必须选择K。
贝叶斯 GMM	椭圆的	最大边际 似然	通过浓度参数自动选择 K	慢；对初始化敏感。
均值-Shift	紧凑的	朝本地 均值 移动	通过 带宽 自动选择 K	可能会很慢。
谱聚类	不规则的	基于图的	处理任何连接的 簇 形状	必须选择 K；无法分配新点；核函数 矩阵代价高。

深度学习

多层感知器

感知器。 建模单个神经元 $y = f(w^T x) = f(\sum_{j=0}^{d} w_j x_j) = \begin{cases} 1 & \text{若} w^\top x \geq 0 \\ -1 & \text{否则} \end{cases}$, 损失： $E(w) = \sum_{i=1}^{N} L(z_i) = \sum_{i=1}^{N} \max(0, -z_i), z_i = y_i w^\top x_i$.



感知器算法 ***/ SGD.*** $w \leftarrow w + \eta y_i x_i = w - \eta g_w(x_i)$, 其中 x_i 被错误分类结果 *(1st 在 ML)*: 1. 仅在可线性分离的数据上收敛； 2. # 迭代 $\sim \frac{1}{m}$, 其中 m 是 类别 之间的分离 (间隔)。
多层感知器现在称为神经网络。

多类别逻辑回归。类别标签 $y \in \{1, \cdots, C\}$, 类别 向量 $\boldsymbol{y} \in \mathbb{R}^C$ 是独热向量。输出 $p(y = j|\boldsymbol{x}) = f_j(\boldsymbol{x}) = s_j(\boldsymbol{g}(\boldsymbol{x})) = \frac{\exp(g_j(\boldsymbol{x}))}{\sum_{k=1}^{C} \exp(g_k(\boldsymbol{x}))}$, $g_j(\boldsymbol{x}) = \boldsymbol{w}_j^T \boldsymbol{x}$, 对 $j \in \{1, \cdots, C\}$.
MLE: $\log p(\boldsymbol{y}|\boldsymbol{x}) = \log \prod_{j=1}^{C} f_j(\boldsymbol{x})^{y_j} = \sum_{j=1}^{C} y_j \log f_j(\boldsymbol{x}) = \boldsymbol{y}^T \log \boldsymbol{f}(\boldsymbol{x})$
$\boldsymbol{W}^* = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \log p(\boldsymbol{y}_i|\boldsymbol{x}_i) = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \boldsymbol{y}_i^T \log \boldsymbol{f}(\boldsymbol{x}_i) = \arg\min_{\boldsymbol{W}} \sum_{i=1}^{N} \{-\sum_{j=1}^{C} y_{ij} \log f_j(\boldsymbol{x}_i)\}$.
交叉熵损失： $L(\boldsymbol{y}, \boldsymbol{f}) = -\sum_{j=1}^{C} y_j \log f_j(\boldsymbol{x})$.
链式法则： $\frac{dL}{d\boldsymbol{w}_j} = \frac{dL}{d\boldsymbol{g}} \frac{d\boldsymbol{g}^T}{d\boldsymbol{w}_j} = \frac{dL}{d\boldsymbol{f}} \frac{d\boldsymbol{f}}{d\boldsymbol{g}} \frac{d\boldsymbol{g}^T}{d\boldsymbol{w}_j} = \boldsymbol{x}(f_j(\boldsymbol{x}) - y_j)$.

多层感知器 ***(MLP).*** 在输入和之间添加隐藏层输出。$\boldsymbol{h} = f(\boldsymbol{W}^T \boldsymbol{x})$, 其中 $f(\cdot)$ 是激活函数。

激活函数： ① Sigmoid: $\mathbb{R} \mapsto [0, 1], f(x) = \frac{1}{1+e^{-x}}$ ② Tanh: $\mathbb{R} \mapsto [-1, 1], f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ③整流线性单元 (ReLU): $f(x) = \max(0, x)$. ④ 注意：ReLU 产生稀疏表示并且训练快速。

MLP - 部分2

过拟合：训练损失下降，但验证损失增加。
早停：当验证损失稳定（变化低于 a）时停止 训练为了防止过拟合，将阈值用于多次迭代。

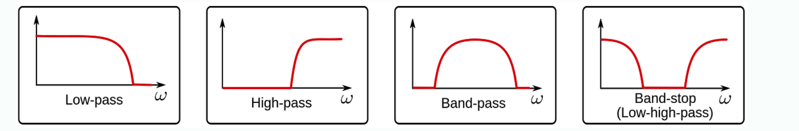
通用逼近定理： A *MLP* 具有一个隐藏层且有限节点可以在所需的误差范围内近似任何连续函数。
梯度消失问题：①含义：反向传播递归地相乘梯度导致数值变小。②原因： 1. 连续乘法 小梯度的 + 大量梯度路径的求和 更深的网络 2. 在某些点饱和的激活函数。

卷积神经网络 (CNNs)

想法：Signal 的 特征 可能具有局部性和 平移，这是不可能的处理 *MLP*. 为了解决这个问题，使用信号中的本地特征提取器。

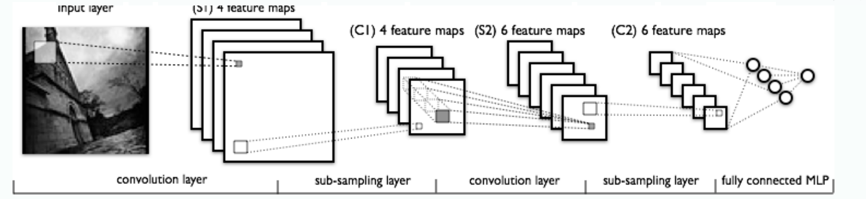
1-D 离散卷积： $s(t) = x * w = \sum_a x(a)w(t - a)$.
2-D 离散卷积： $s(n, m) = x * w = \sum_a \sum_b x(a, b)w(n - a, m - b)$.
属性： $s = x * w \Rightarrow s(t - a) = x(t - a) * w(t)$. 注意：卷积操作等同于频域中的乘法。

解释 **1:** w 是信号频谱上的一个滤波器 x, e.g. 低通，高通，带通，低高通，移动平均。

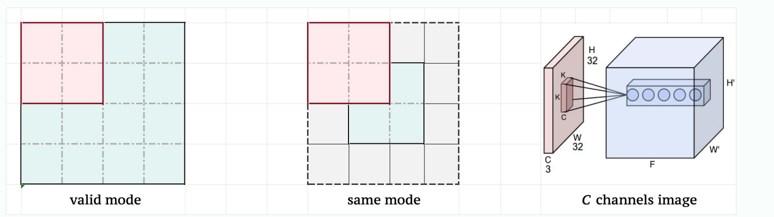


①变换到频域 $X(w)$ 由 *DFT* $X(\omega) = \sum_t x(t)e^{-i\omega t}$ 其中 $e^{-i\omega t} = \cos(\omega t) - i\sin(\omega t)$ ②按……进行筛选w ③回到时域 $x(t) = \frac{1}{2\pi} \int X(\omega)e^{i\omega t}d\omega$. 然后 $s(t) = x(t) * w(t) \Longleftrightarrow S(\omega) = X(\omega)W(\omega)$.
解释 **2:** w 是一个模式（模板），并尝试找到这个模式 i.e. 模式w 匹配本地 $x \Rightarrow x = \frac{w}{||w||}$.

***CNNs*:** 一系列卷积层、子采样层以及 MLP 分类器。



2D 卷积滤波器： $h = f(\sum_{x,y} W_{x,y} P_{x,y})$, 其中 W: 模板 P: 图像的一部分
卷积模式：①有效模式：全部 $P_{x,y}$ 具有有效的图像值 → 缩小 $(k - 1)/2$ ②相同模式：对图像边界进行零填充 → 与……同样大小输入图像。



C 通道 图像： $C \times H \times W \xrightarrow[C \times F \times K \times K]{F \text{ 卷积滤波器}} F$ 通道 特征： $F \times H' \times W'$
空间子采样：①步幅 $(s) \rightarrow (m - k_x + 1 - s) \times (n - k_y + 1 - s)$ ② 最大池化 $\rightarrow W_{x,y}P_{x,y} = \max(P_{x,y})$, 引入平移不变性（稳健）
③效果：降低特征地图的空间分辨率，从而提高推理速度。
感受野：输入中的哪些像素会影响特定节点 $\rightarrow k_{i-1} + k_i - 1$
优点：①在整个图像中提取相同的特征 ②池化是对变化具有鲁棒性 ③比……少参数 *FCNN*: $C \times H \times W \rightarrow F \times H \times W$, *FCNN* 是$(CHW + 1) \times (FHW)$ 且 *CNN* 是$(CKK + 1) \times F$

正则化 方法

L2 正则化： $\hat{L} = L + \lambda\frac{1}{2}||\boldsymbol{w}||^2$. *SGD*: $\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \eta\frac{d\hat{L}}{d\boldsymbol{w}_k} = \boldsymbol{w}_k - \eta(\frac{dL}{d\boldsymbol{w}_k} + \lambda\boldsymbol{w}_k) = (1 - \eta\lambda)\boldsymbol{w}_k - \eta\frac{dL}{d\boldsymbol{w}_k}$, 这也被称为权重衰变。
注意:① *L2* 正则化 应用于所有层。因为在应用时 L2 正则化 仅在一层上，另一层可以调整以获得相同网络： $f(\boldsymbol{x}) = \boldsymbol{A}^T r(\boldsymbol{B}^T\boldsymbol{x}) = \boldsymbol{A}^T r(\frac{\epsilon}{\epsilon}\boldsymbol{B}^T\boldsymbol{x}) = \frac{1}{\epsilon}\boldsymbol{A}^T r(\epsilon\boldsymbol{B}^T\boldsymbol{x})$. 等同假设在权重上使用高斯 先验 MAP 估计。

提前停止：限制训练迭代次数（进行多远 $\boldsymbol{w}$ 可以移动)
装袋法：结合多个模型的预测（模型平均）。
结合 K 误差为…的模型 ϵ_i 且 $\text{MSE}_i = \mathbb{E}[\epsilon_i^2] = V$, 然后 $\text{MSE}_c = \frac{V}{K} + \frac{K-1}{K}C$, 其中 $C = \mathbb{E}[\epsilon_i\epsilon_j]$ 是第 i 个的错误与…之间的相关性第 j 个模型。因此MSE 减少时 $C = 0$.

对 *NN*, 可以使用相同的数据集，但初始化不同 / 超参数。
Dropout：①目标：通过随机方式近似大量模型的集合移除输入 /以某种概率隐藏节点。②训练： (1) 掉以概率“out”每个节点 p (2) 训练 每次迭代中的简化网络
③测试：使用所有节点进行预测并按比例调整输出 $1 - p$ ④效果：(1) 在计算梯度时阻塞某些路径 → 所以减少消失梯度问题 (2) 减少预测误差 (3) 减少有效容量网络的 (4) 提高分类器的鲁棒性（不仅仅依赖一个特征）。

数据增强：人工排列数据，使用 平移，翻转，旋转、变形、添加像素噪声、缩放、颜色、亮度、对比度等制造 1. 数据集更大 / 模型看到了更多的数据变体， 2. 这对置换不变的网络。

如何深入?

我们需要多少样本? 假设 $\mathcal{N}$ 是一个前馈网络含 W 权重， L 层，使用固定的分段激活函数。然后 VC维数$(\mathcal{N}) = O(WL\log W + WL^2)$. （Bartlett、Maiorov、Meir、1998)
VC 维度是函数类别容量和样本的统计测量尺寸需要按比例缩放。参数数量相同时，更深的网络需要更多的数据 (WL^2).

修正线性单元 $\text{ReLU}(z) = \max(0, z)$. 优势：①简便快速训练。②稀疏表示， i.e. 大多数节点将输出零，减少梯度消失问题

批量归一化。①为什么？激活分布会发生变化 1 层，那么所有层都会受到影响，需要改变。②批量归一化：对于每个节点在每一层，对输出进行归一化到 $\hat{y}_i = \frac{y_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ 在每个小批次上 → 缩放与平移 到$z_i = \gamma\hat{y}_i + \beta$, 其中 γ, β 是可以学习的。③效果：减少……的影响协变量偏移 → 加速 训练 + 稳定梯度 + 更好的泛化（不需要丢弃） / L_2-reg). ④注意：最好放在线性层之后。

学习率计划。①线性变化： $\eta_k = (1 - \frac{k}{T})\eta_0 + \frac{k}{T}\eta_T$. ②衰变： $\eta_k = \frac{1}{1+\delta k}\eta_0$, 其中 $0 < \delta < 1$. ③自适应计划：减少 *lr* 当验证损失不再改善。

动量 在小批量中保持梯度的滚动平均值：
$\boldsymbol{v}^{(t)} = \alpha\boldsymbol{v}^{(t-1)} - \eta\frac{dL}{d\boldsymbol{w}}, \boldsymbol{w}^{(t)} \leftarrow \boldsymbol{w}^{(t-1)} + \boldsymbol{v}^{(t)}$.

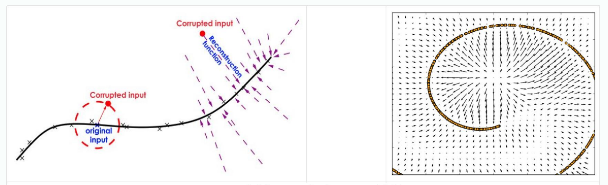
Transformer 学习 & 微调

Transformer 学习。与其重新训练整个预训练网络， ① 修复提取 特征 的下层 ②训练 最后的几层以执行新任务。 `*list(model.children())[:-1]`

Auto-Encoders

Auto-Encoder (AE). ① *Goal:* use the hidden layer to learn the lower-dimensional latent representation of the data ② *Method:* encoder $\boldsymbol{z} = f(\boldsymbol{x}) = h_1(\boldsymbol{A}^T\boldsymbol{x}) \rightarrow$ decoder $\hat{\boldsymbol{x}} = g(\boldsymbol{z}) = g(f(\boldsymbol{x})) = h_2(\boldsymbol{B}^T\boldsymbol{z})$ ③ *Target:* minimize the difference between $\boldsymbol{x}$ and $\hat{\boldsymbol{x}}$: 1. cross-entropy: $\boldsymbol{x}\log\hat{\boldsymbol{x}} + (1-\boldsymbol{x})\log(1-\hat{\boldsymbol{x}})$ 2. MSE: $||\boldsymbol{x}-\hat{\boldsymbol{x}}||^2$ ④ *Weight Sharing:* decoder's weights are the transpose of encoder's weights, $\boldsymbol{A}^T = \boldsymbol{B}$. ⑤ *Note:* if h_1, h_2 are linear then it's similar to PCA ⑥ *Better:* 1. regularization like $L = ||\boldsymbol{x} - g(f(\boldsymbol{x}))||^2 + \lambda||f(\boldsymbol{x})||_1$ 2. denoising.

Denoising Auto-Encoder (DAE). ① *Problem of AE:* minimum occurs when $g(f(\boldsymbol{x})) = \boldsymbol{x}$ means learn nothing ②*Idea:* add noise to corrupt the inputs to make encoder learn more about the data manifold: minimize $||\boldsymbol{x} - g(f(\tilde{\boldsymbol{x}}))||^2$ ③ *Interpretation (1):* DAE is learning a **vector field** from the corrupted $\tilde{\boldsymbol{x}}$ to the data manifold (clean $\boldsymbol{x}$) ③ *Interpretation (2):* learn a reconstruction distribution $p_{\text{recon}}(\boldsymbol{x}|\tilde{\boldsymbol{x}})$: sample training example $\boldsymbol{x} \rightarrow$ add noise $\tilde{\boldsymbol{x}} \rightarrow$ maximize $p_{\text{recon}}(\boldsymbol{x}|\tilde{\boldsymbol{x}}) = p_{\text{decoder}}(\boldsymbol{x}|\boldsymbol{z}), \boldsymbol{z} = f(\tilde{\boldsymbol{x}}) \rightarrow$ *MLE:* $\min\sum_i ||\boldsymbol{x}_i - g(f(\tilde{\boldsymbol{x}}_i))||^2$ (assume $p_{\text{decoder}}(\boldsymbol{x}|\boldsymbol{z})$ is Gaussian)



vector field towards data manifold

Convolutional Auto-Encoder. encoder - a standard *CNN* w/o classifier → decoder - the opposite architecture, replace max-pooling with upsampling.

Deep Generative Models

For data-label pair (x, z), forward generative process: $p(x|z)$ and discriminative classifier: $p(z|x)$. We want *NN* to be generative model (multiple outputs). ① *Idea:* use *NN* to predict the parameters of a distribution, e.g. $y \sim \mathcal{N}(\mu(\boldsymbol{x}), \sigma(\boldsymbol{x})^2)$ ② *Reparameterization:* decompose y into parameters and "randomness", e.g. $y = \mu(\boldsymbol{x}) + \sigma(\boldsymbol{x})z, z \sim \mathcal{N}(0,1)$. In general $y \sim p(y|x) \Rightarrow y = f(z, x), z \sim p(z)$ where x are the inputs, z is the source of randomness, f transforms samples of z to samples of y which modeled as a *NN* and learned. ③ *Note:* Computing $p(y)$ requires computing the inverse of the transformation f, learning f is equal to learn the inverse CDF of y.

Variational AutoEncoder (VAE). ① *Problem of AE:* decoder never sees encoded latent vectors outside of the training set thus hard to decode new images. ② *Idea:* introduce noise in the latent vectors ③ *Model:* let latent variable $\boldsymbol{z}$, observation $\boldsymbol{x}$, prior $p(\boldsymbol{z}) \sim \mathcal{N}(0, \boldsymbol{I})$, likelihood $p(\boldsymbol{x}|\boldsymbol{z})$, loss function $-\log p(\boldsymbol{x}|\boldsymbol{z})$; given $\boldsymbol{x}_i$ to infer (posterior distribution) $\boldsymbol{z}_i$: $q(\boldsymbol{z}_i|\boldsymbol{x}_i) \sim \mathcal{N}(\mu(\boldsymbol{x}_i), \text{diag}(\sigma(\boldsymbol{x}_i)^2))$ ($\mu(\cdot), \sigma(\cdot)$ are *NN* outputs) ④ *Reparameterization:* $\boldsymbol{z}_{il} = \mu(\boldsymbol{x}_i) + \sigma(\boldsymbol{x}_i) \circ \epsilon_{il}$ where $\epsilon_{il} \sim \mathcal{N}(0, \boldsymbol{I})$ ⑤ *Note:* decoder and encoder don't share weights ⑥ *VAE loss:*

$$L(\boldsymbol{x}_i) = \underbrace{-\frac{1}{L}\sum_{l=1}^{L}\log p(\underbrace{\boldsymbol{x}_i|\boldsymbol{z}_{il}}_{\text{decoder}})}_{\text{reconstruction loss}} + \underbrace{KL(q(\boldsymbol{z}|\boldsymbol{x}_i), p(\boldsymbol{z}_i))}_{\text{regularizer}},$$

where $\boldsymbol{z}_{il} = \mu(\boldsymbol{x}_i) + \sigma(\boldsymbol{x}_i) \circ \epsilon_{il}$, $\epsilon_{il} \sim \mathcal{N}(0, \boldsymbol{I})$(encoder).
Convolutional VAE. replace the Dense layers with Conv2D and Pooling.
Comparison with AE. ① *Similarities:* 1. aim to encode the input into a latent space, and then decode it to reconstruct the original input 2. use reconstruction loss (MSE, cross-entropy) for training. ② *Differences:* 1. *AE* predicts a single latent vector z, while *VAE* predicts a Gaussian posterior distribution of the latent vector z 2. For training, *VAE* also has a regularization term that keeps the posterior near to the prior distribution 3. *VAE* samples the latent vector z during training, according to the posterior. ③ *Advantage: VAE* samples from the latent space, it sees more latent vectors during training, which prevents the model from overfitting to each latent vector. This helps interpolation between the latent vectors, creating a smoother output space.

Deep Generative Models – Generative Adversarial Networks

① *Goal:* generate samples from a probability density $p_{data}(x)$ which is represented by a training set $\{x_i\}$. ② *Setup:* true data samples: $x \sim p_d(x)$ but only have samples $\mathcal{X} = \{x_i\}$; generated samples: $x \sim p_m(x)$ ③ *Model:* reparam $x = g(z)$, $z \sim N(0, I)$ where $g(\cdot)$ is a *NN* → max $p_m(x)$ ④ *Problem:* cannot explicitly compute $p_m(x)$ due to *NN* ⑤ *Solution:* look at the samples directly → build a classifier to predict whether a sample is *real* or *fake* (generated).

GAN. ① *Architecture: Generator (G)* - make fake samples to fool the discriminator ↔ *Discriminator (D)* - classify x as real or fake ② *Model:* define "real" ($y = 1$) and "fake" ($y = 0$), discriminator (classifier) $D(x) = p(y = 1|x)$ ③ *Train:* D: $\min_D -\sum_i y_i \log D(x_i) + (1-y_i)\log(1-D(x_i)) = \max_D[\mathbb{E}_{x\sim p_d}[\log D(x_i)] + \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\log(1-D(G(z)))]]$; G to fool D → final training problem: $\min_G \max_D \mathbb{E}_{x\sim p_d}[\log D(x_i)] + \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\log(1-D(G(z)))]$. ④ *Note:* 1. *GANs* don't provide an explicit likelihood estimation, but *VAEs* do. 2. training *GAN* is hard due to its adversarial nature.

Analysis. ① *Optimal D:* $D^*(x) = \frac{p_d(x)}{p_d(x)+p_m(x)} = p(\text{real}|x)$ ② *Optimal G:* $p_m^*(x) = p_d(x)$ ③ *Advantage:* don't need to approximate the likelihood function; don't need the partition function; easy to generate samples ④ *Lack:* cannot compute the likelihood value (probability density) of a sample; hard to train ④ *Mode collapse problem:* when $p_d(x)$ have more than 2 modes, G samples from one mode → D catches this mode → G switches to other mode → $\cdots$.
Application: Style transfer (CycleGAN), Face Synthesis . . ..

Diffusion Model. ① *Goal:* define forward diffusion process gradually turns image into noise → learn reverse process gradually remove noise and recover image ② *Forward diffusion process:* $\boldsymbol{x}_t = \sqrt{1-\beta_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, \boldsymbol{I})$ where $\boldsymbol{x}_0$ = image and $\{\beta_1, \cdots, \beta_T\}$ is *noise variance schedule*, $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t|\sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\boldsymbol{I}) \rightarrow q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) = \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ ③ *Reverse diffusion process:* $p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}|\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \Sigma_\theta(\boldsymbol{x}_t, t)) \rightarrow p(\boldsymbol{x}_{0:T}) = p_\theta(\boldsymbol{x}_T)\prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ ④ *Train:* $\mathbb{E}[-\log p_\theta(\boldsymbol{x}_0)] \leqslant \mathbb{E}_q\left[-\log\frac{p_\theta(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0)}\right] = \sum_{t=0}^{T} L_t \rightarrow \min\sum_{t=0}^{T} L_t$ where $L_0 = -\log p_\theta(\boldsymbol{x}_0|\boldsymbol{x}_1), L_{t-1} = KL(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0), p(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)), L_T = KL(q(\boldsymbol{x}_T|\boldsymbol{x}_0, p(\boldsymbol{x}_T))$.

Model	Description
Autoencoder	Unsupervised dimensionality reduction and clustering.
Convolutional Autoencoder	AE for images.
Masked Autoencoder (MAE)	Self-supervised representation learning.
Variational Autoencoder (VAE)	Improve interpolation ability, generative model.
Generative Adversarial Networks (GAN)	Learn to generate samples from a distribution.
Diffusion Model	Transform noise into image, step by step.

ResNet

The shortest path of ResNet contains most of the gradient signal.
A ResNet with k residual blocks can be viewed as equivalent to an ensemble of 2^k networks.

Training Problems

Overfitting. Try regularization, weight decay, early stopping, dropout, ensembling, data augmentation, (max-pooling, increase channels, decrease kernel size)
Generalization gap. Try regularization, data augmentation, and dropout, to improve generalization
Underfitting. Try training for more epochs, or increase the learning rate.

Large Language Models

Tokenization. Split natural language input into a sequence of subwords (tokens). ① *Rule-based:* Split by Spaces + Special Handling of Punctuations, default for English, see NLTK ② *Dictionary/Model-based:* for languages that there are no easy rules to split the sentence, e.g. jieba for Chinese ③ *Statistics-based:* most widely used in NLP, e.g. Byte-pair encoding (BPE), repeat the process of merging most frequently appearing nearby subtokens.

Langauge Models (LMs). ① *Task:* build a probablistic model $p(T)$ for the input tokens sequence $T = [t_0, t_1, ..., t_{N-1}]$ ② *Decomposition:* $p(T) = p(t_0) * p(t_1|t_0) * p(t_2|t_0, t_1) * ... * p(t_{N-1}|t_0, t_1, ..., t_{N-2}) \rightarrow$ next token prediction: $p(t_i|t_0, ..., t_{i-1})$.
LM evaluation. ① *perplexity (PPL):* $PPL = b^{-\frac{1}{N}\sum_x p(x)\log_b p(x)}$ ② *2-based PPL:* $PPL = 2^{-\frac{1}{N}\sum_i \log_2 p(t_i)}$
Next Token Prediction. ① *N-gram:* assume $p(t_i|t_0, ..., t_{i-1}) \approx p(t_i|t_{i-n}, ..., t_{i-1})$, e.g. Unigram: $p(t_i)$, Bi-gram: $p(t_i|t_{i-1})$, Tri-gram: $p(t_i|t_{i-2}, t_{i-1})$, ... ② *N-gram problem:* strong assumption, sparsity with OOV words, semantic meaning lack ③ *Neural LM:* utilizes a vector to represent each word/token to allow better modeling of the semantics, e.g. *word embedding*, *word2vec*, *Glove* ④ *Transformer LM:* attention-based, easy to parallesize and can scale.

Large Language Models (LLMs). *Scaling Laws:* ① ELmo and BERT (2018) w/ Large-scale Pre-training → ② GPT-3 (2020) w/ In-Context Learning ③ InstructGPT (2022) w/ RLHF → ③ OpenAI-o1 (2024) and Deepseek-R1 (2025) w/ Reasoning.

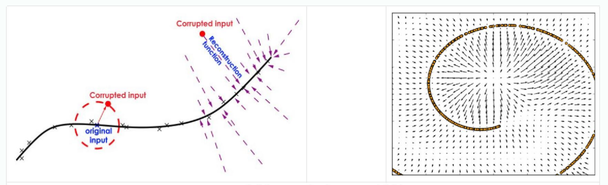
Vision Language Models (VLM). Region-Based CNN (RCNN / R-CNN) → Fast R-CNN → Faster R-CNN → Region Proposal Network (RPN) → You Only Look Once (YOLO) → DEtection TRansformer (DETR) . . ..

小红书 小红书号：Ray0917

自编码器

自编码器 ***(AE).*** ①目标：使用隐藏层来学习低维 数据的潜在表示 ②方法：编码器 $\boldsymbol{z} = f(\boldsymbol{x}) = h_1(\boldsymbol{A}^T\boldsymbol{x}) \rightarrow$ 解码器 $\hat{\boldsymbol{x}} = g(\boldsymbol{z}) = g(f(\boldsymbol{x})) = h_2(\boldsymbol{B}^T\boldsymbol{z})$ ③目标：最小化……之间的差异 $\boldsymbol{x}$ 且 $\hat{\boldsymbol{x}}$: 1. 交叉熵：$\boldsymbol{x}\log\hat{\boldsymbol{x}} + (1-\boldsymbol{x})\log(1-\hat{\boldsymbol{x}})$ 2. MSE: $||\boldsymbol{x}-\hat{\boldsymbol{x}}||^2$ ④权重 共享: 解码器 的 权重 是转置编码器的权重，$\boldsymbol{A}^T = \boldsymbol{B}$. ⑤注意：如果$h_1, h_2$ 是线性的，那么它类似于 PCA ⑥更好：1. 正则化 喜欢 $L = ||\boldsymbol{x} - g(f(\boldsymbol{x}))||^2 + \lambda||f(\boldsymbol{x})||_1$ 2. 去噪。

去噪 自编码器 ***(DAE).*** ①……的问题 *AE:* 最小值出现于 $g(f(\boldsymbol{x})) = \boldsymbol{x}$ 均值 什么也不学 ②想法：加入噪声来破坏输入让编码器更多地了解数据流形：最小化 $||\boldsymbol{x} - g(f(\tilde{\boldsymbol{x}}))||^2$ ③解释 *(1):* DAE 正在从损坏的数据中学习一个向量场 $\tilde{\boldsymbol{x}}$ 至数据流形（干净 $\boldsymbol{x}$) ③解释 *(2):* 学习重建分配 $p_{\text{recon}}(\boldsymbol{x}|\tilde{\boldsymbol{x}})$：示例 训练 例子 $\boldsymbol{x} \rightarrow$ 添加噪音 $\tilde{\boldsymbol{x}} \rightarrow$ 最大化 $p_{\text{recon}}(\boldsymbol{x}|\tilde{\boldsymbol{x}}) = p_{\text{decoder}}(\boldsymbol{x}|\boldsymbol{z}), \boldsymbol{z} = f(\tilde{\boldsymbol{x}}) \rightarrow$ *MLE:* $\min\sum_i ||\boldsymbol{x}_i - g(f(\tilde{\boldsymbol{x}}_i))||^2$（假设 $p_{\text{decoder}}(\boldsymbol{x}|\boldsymbol{z})$ 是 高斯)



vector field towards data manifold

卷积 自编码器。编码器 - 一个标准 *CNN* w/o 分类器 → 解码器- 相反的架构，将最大池化替换为上采样。

深度 生成式 模型

用于数据-标签对 (x, z), 前向生成式过程：$p(x|z)$ 和 判别式 分类器：$p(z|x)$. 我们想要*NN* 成为 生成式 模型（多输出）。①想法：使用*NN* 预测分布的参数，e.g. $y \sim \mathcal{N}(\mu(\boldsymbol{x}), \sigma(\boldsymbol{x})^2)$ ② 重参数化：分解 y 到参数和 " 随机性 e.g. $y = \mu(\boldsymbol{x}) + \sigma(\boldsymbol{x})z, z \sim \mathcal{N}(0,1)$. 一般来说 $y \sim p(y|x) \Rightarrow y = f(z, x), z \sim p(z)$ 其中 x 是输入，z 是随机性的来源，f 转换样本 z 到样本 y 被建模为一个 *NN* 并学会了。③注意：计算 $p(y)$ 需要计算变换的逆 f, 学习 f 等于学习逆操作 CDF 的y.

变分的 ***AutoEncoder (VAE).*** ①……的问题*AE:* 解码器 从不显示编码位于训练集合之外的潜在向量因此难以解码新图像。② 想法：在潜在向量中引入噪声 ③模型：让潜在变量 $\boldsymbol{z}$, 观察 $\boldsymbol{x}$, 先验 $p(\boldsymbol{z}) \sim \mathcal{N}(0, \boldsymbol{I})$, 似然 $p(\boldsymbol{x}|\boldsymbol{z})$, 损失函数 $-\log p(\boldsymbol{x}|\boldsymbol{z})$; 给定 $\boldsymbol{x}_i$ 推断（后验 分布）$\boldsymbol{z}_i$: $q(\boldsymbol{z}_i|\boldsymbol{x}_i) \sim \mathcal{N}(\mu(\boldsymbol{x}_i), \text{diag}(\sigma(\boldsymbol{x}_i)^2))$ ($\mu(\cdot), \sigma(\cdot)$ 是 *NN* 输出）④重新参数化：$\boldsymbol{z}_{il} = \mu(\boldsymbol{x}_i) + \sigma(\boldsymbol{x}_i) \circ \epsilon_{il}$ 其中 $\epsilon_{il} \sim \mathcal{N}(0, \boldsymbol{I})$ ⑤ 注意：解码器 和 编码器 不共享 权重 ⑥ *VAE* 损失:

$$L(\boldsymbol{x}_i) = \underbrace{-\frac{1}{L}\sum_{l=1}^{L}\log p(\underbrace{\boldsymbol{x}_i|\boldsymbol{z}_{il}}_{\text{解码器}})}_{\text{重建损失}} + \underbrace{KL(q(\boldsymbol{z}|\boldsymbol{x}_i), p(\boldsymbol{z}_i))}_{\text{正则化器}},$$

其中 $\boldsymbol{z}_{il} = \mu(\boldsymbol{x}_i) + \sigma(\boldsymbol{x}_i) \circ \epsilon_{il}$, $\epsilon_{il} \sim \mathcal{N}(0, \boldsymbol{I})$(编码器).
卷积 ***VAE.*** 将全连接层替换为卷积层 2D 以及池化。
与……比较 ***AE.*** ①相似点：1. 旨在将输入编码为潜在表示空格，然后解码以重建原始输入 2. 使用重建损失(MSE, 交叉熵)用于训练。②差异：1. *AE* 预测一个潜在向量 z, 当 *VAE* 预测一个 高斯 后验 分布的潜在向量 z 2. 对于 训练，*VAE* 还有一个 正则化 项保持接近先验分布的后验 3. *VAE* 对潜在向量进行采样 z 在训练期间，根据后验。③ 优势：*VAE* 来自……的样本潜在空间，它在 训练 期间看到更多潜在向量，这防止了模型对每个潜在向量的过拟合。这有助于在它们之间进行插值潜在向量，创建一个更平滑的输出空间。

深度 生成式 模型 – 生成式 对抗网络

①目标：从概率密度生成样本 $p_{data}(x)$ 哪一个是由 训练 集表示 $\{x_i\}$. ②设置：真实数据样本：$x \sim p_d(x)$ 但是只有有样品 $\mathcal{X} = \{x_i\}$; 生成的样本：$x \sim p_m(x)$ ③型号：reparam $x = g(z)$, $z \sim N(0, I)$ 其中 $g(\cdot)$ 是 *NN* → max $p_m(x)$ ④问题：不能明确计算 $p_m(x)$ 由于 *NN* ⑤解决方案：直接查看样本 → 构建一个分类器来预测样本是真实的还是伪造的（生成的）。

GAN. ①建筑学：生成器 *(G)* - 制作假样本来欺骗判别器 ↔ 判别器 *(D)* - 分类 x 无论是真还是假 ②模型：定义“真实”($y = 1$) 和“假”($y = 0$), 判别器 (分类器) $D(x) = p(y = 1|x)$ ③训练：D: $\min_D -\sum_i y_i \log D(x_i) + (1-y_i)\log(1-D(x_i)) = \max_D[\mathbb{E}_{x\sim p_d}[\log D(x_i)] + \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\log(1-D(G(z)))]]$; G 愚弄 D → 最终训练 问题：$\min_G \max_D \mathbb{E}_{x\sim p_d}[\log D(x_i)] + \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\log(1-D(G(z)))]$. ④注意：1. GAN 不提供显式的 似然 估计，但 VAE 提供。2. 训练 *GAN* 由于其对抗性特征而很难。

分析。①最佳 *D:* $D^*(x) = \frac{p_d(x)}{p_d(x)+p_m(x)} = p(\text{真实}|x)$ ②最佳 *G:* $p_m^*(x) = p_d(x)$ ③优点：不需要逼近似然函数；不需要分区函数；容易生成样本 ④缺点：无法计算一个样本的 似然 值（概率密度）；很难 训练 ④模式坍塌问题：何时 $p_d(x)$ 有超过 2 模式，G 来自一个的样本模式 → D 捕捉这一模式 → G 切换到其他模式 → $\cdots$.
应用：风格迁移（CycleGAN）、人脸合成 . . ..

扩散模型 ①目标：定义正向扩散过程逐渐将图像变成噪声 → 逐渐学习逆过程移除 噪音 且 恢复 图像 ② 前进 扩散 过程：$\boldsymbol{x}_t = \sqrt{1-\beta_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, \boldsymbol{I})$ 其中 $\boldsymbol{x}_0$ = 图像和 $\{\beta_1, \cdots, \beta_T\}$ 是 噪音 方差 时间表 $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t|\sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\boldsymbol{I}) \rightarrow q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) = \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ ③反向扩散过程：$p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}|\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \Sigma_\theta(\boldsymbol{x}_t, t)) \rightarrow p(\boldsymbol{x}_{0:T}) = p_\theta(\boldsymbol{x}_T)\prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ ④ 训练：$\mathbb{E}[-\log p_\theta(\boldsymbol{x}_0)] \leqslant \mathbb{E}_q\left[-\log\frac{p_\theta(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0)}\right] = \sum_{t=0}^{T} L_t \rightarrow \min\sum_{t=0}^{T} L_t$ 其中 $L_0 = -\log p_\theta(\boldsymbol{x}_0|\boldsymbol{x}_1), L_{t-1} = KL(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0), p(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)), L_T = KL(q(\boldsymbol{x}_T|\boldsymbol{x}_0, p(\boldsymbol{x}_T))$.

模型	描述
自编码器	无监督的降维和聚类。
卷积 自编码器	AE 用于图片。
掩码 自编码器 (MAE)	自监督表示学习。
变分 自编码器 (VAE)	提高插值能力，生成式 模型。
生成式 对抗网络 (GAN)	学习从分布中生成样本。
扩散模型	逐步将噪声转化为图像。

ResNet

的最短路径 ResNet 包含大部分梯度信号。
A ResNet 含 k 残差块可以被视为等同于一个集成的2^k 网络。

训练 问题

过拟合。尝试 正则化、权重 衰减、提前停止、丢弃法，集成，数据增强，（最大池化，增加 通道，减少 核函数大小）
泛化差距。尝试正则化、数据增强和 dropout，以提高泛化能力
欠拟合。尝试 训练 以获得更多 训练轮次，或者增加学习率。

大型语言模型

分词。将自然语言输入拆分为子词序列（词元). ①基于规则：按空格分割 + 标点符号的特殊处理，英语的默认设置，请参见 NLTK② 词典 /基于模型 : 对于存在的语言没有简单的规则来分割句子，e.g. 结巴中文分词 ③基于统计的方法：在自然语言处理领域中使用最广泛，e.g. 字节对编码（BPE），重复该过程合并附近最频繁出现的子标记。

语言模型 (LMs)。①任务：建立一个概率模型 $p(T)$ 对输入 词元 序列 $T = [t_0, t_1, ..., t_{N-1}]$ ②分解：$p(T) = p(t_0) * p(t_1|t_0) * p(t_2|t_0, t_1) * ... * p(t_{N-1}|t_0, t_1, ..., t_{N-2}) \rightarrow$ 下一个 词元 预测：$p(t_i|t_0, ..., t_{i-1})$.
LM 评估。①困惑度 *(PPL):* $PPL = b^{-\frac{1}{N}\sum_x p(x)\log_b p(x)}$ ② *2-based PPL:* $PPL = 2^{-\frac{1}{N}\sum_i \log_2 p(t_i)}$
下一个 词元 预测。①N-gram：假设 $p(t_i|t_0, ..., t_{i-1}) \approx p(t_i|t_{i-n}, ..., t_{i-1})$, e.g. 单字词：$p(t_i)$, 二元组：$p(t_i|t_{i-1})$, 三元组：$p(t_i|t_{i-2}, t_{i-1})$, ... ②N-gram 问题：强假设、OOV词的稀疏性、语义缺失 ③神经 *LM:* 使用向量来表示每个单词 /词元 以允许更好语义建模，e.g. 词嵌入，词 2向量, GloVe ④Transformer *LM:* 基于注意力的，易于并行化且可扩展。

大型语言模型（LLMs）。规模法则：①ELmo 和 BERT (2018) w/ 大规模预训练 → ② GPT-3 (2020) w/ 上下文学习 ③ InstructGPT (2022) w/ RLHF → ③ OpenAI-o1 (2024) 和 Deepseek-R 1 (2025) w/ 推理

视觉语言模型 ***(VLM).*** 基于区域 CNN (RCNN / R-CNN) → 快速 R-CNN → 更快的 R-CNN → 区域建议网络 (RPN) → 你一次性看 (YOLO) → 检测 Transformer (DETR) . . ..

小红书 小红书号：Ray0917

Algorithm 1 Orthogonal Matching Pursuit (OMP)

```
Initialize the residual: $\boldsymbol{r} = \boldsymbol{y}$
for $t = 1$ to $K$ do
    Find the most correlated feature: $j = \arg\max_j |r^T \boldsymbol{x}_j|$, where $\boldsymbol{x}_j$ is the $j$-th row of $\boldsymbol{X}$ (the $j$-th feature).
    Compute the weight: $\boldsymbol{w}_j = \arg\min_{w_j} \|\boldsymbol{r} - \boldsymbol{x}_j \boldsymbol{w}_j\|^2$
    Update the residual: $\boldsymbol{r} = \boldsymbol{r} - \boldsymbol{x}_j \boldsymbol{w}_j$
end for
```

Algorithm 2 RANSAC (Random Sample Consensus)

```
Given: training set $D = \{x_i, y_i\}$, threshold $\epsilon$, loss function $L = \sum l(x, y)$
Classify all data as inlier or outlier by calculating the prediction errors $l$ and comparing to the threshold $\epsilon$. (typically set as the median absolute deviation (MAD) of $y$, i.e. $\mathrm{median}(|y_i - \mathrm{median}(y)|)$)
The set of inliers is called the consensus set.
Save the model with the highest number of inliers.
Use the largest consensus set to learn the final model.
```

Algorithm 3 *k*-means Clustering

```
Input: Data points $\{\boldsymbol{x}_i\}_{i=1}^n \subset \mathbb{R}^d$, number of clusters $k$.
Initialization: Choose initial cluster centers $\{\boldsymbol{c}_j^{(0)}\}_{j=1}^k \subset \mathbb{R}^d$.
Set iteration counter $t \leftarrow 0$.
repeat
    Assignment step: For each $i = 1, \ldots, n$, update the cluster label $z_i^{(t)} \leftarrow \arg\min_{j \in \{1,\ldots,k\}} \|\boldsymbol{x}_i - \boldsymbol{c}_j^{(t)}\|_2^2$.
    Update step: For each $j = 1, \ldots, k$, update the cluster center $\boldsymbol{c}_j^{(t+1)} \leftarrow \frac{1}{|\mathcal{C}_j^{(t)}|} \sum_{i \in \mathcal{C}_j^{(t)}} \boldsymbol{x}_i, \quad \mathcal{C}_j^{(t)} := \{\, i \in \{1, \ldots, n\} : z_i^{(t)} = j \,\}$.
    $t \leftarrow t + 1$.
until Convergence.
```

Algorithm 4 EM Algorithm for Gaussian Mixture Model (GMM)

```
Input: Data points $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^d$, number of clusters $K$, initial parameters for means $\{\boldsymbol{\mu}_j\}_{j=1}^K$, covariances $\{\Sigma_j\}_{j=1}^K$, and weights $\{\pi_j\}_{j=1}^K$.
Set iteration counter $t \leftarrow 0$.
repeat
    E-step: Calculate cluster membership for each point $\boldsymbol{x}_i$: $\hat{z}_{ij} = p(z_i = j|\boldsymbol{x}_i) = \frac{\pi_j \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_j, \Sigma_j)}{\sum_{k=1}^K \pi_k \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_k, \Sigma_k)}$.
    M-step: Update the parameters of each Gaussian cluster: Soft count of points in cluster $j$: $N_j = \sum_{i=1}^N \hat{z}_{ij}$.
    $\pi_j \leftarrow \frac{N_j}{N}, \boldsymbol{\mu}_j \leftarrow \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} \boldsymbol{x}_i, \Sigma_j \leftarrow \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} (\boldsymbol{x}_i - \boldsymbol{\mu}_j)(\boldsymbol{x}_i - \boldsymbol{\mu}_j)^\top$.
    $t \leftarrow t + 1$.
until Convergence.
```

Algorithm 1 Mean-shift Algorithm

```
Input: Data points $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^d$, bandwidth parameter $h$, initial point $\boldsymbol{x}^{(0)}$.
Set iteration counter $t \leftarrow 0$.
repeat
    1) Find the neighbors of $\boldsymbol{x}^{(t)}$: For each data point $\boldsymbol{x}_i$, calculate the distance to $\boldsymbol{x}^{(t)}$. Find the set of neighbors within a bandwidth radius $h$: $\mathcal{N}(\boldsymbol{x}^{(t)}) = \left\{ \boldsymbol{x}_i : \|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2 \le h \right\}$.
    2) Shift the point towards the mean of the neighbors: Update $\boldsymbol{x}^{(t+1)}$ to the weighted mean of the points in $\mathcal{N}(\boldsymbol{x}^{(t)})$ using the Gaussian kernel:
        $$\boldsymbol{x}^{(t+1)} = \frac{\sum_{\boldsymbol{x}_i \in \mathcal{N}(\boldsymbol{x}^{(t)})} \boldsymbol{x}_i \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2^2}{2h^2}\right)}{\sum_{\boldsymbol{x}_i \in \mathcal{N}(\boldsymbol{x}^{(t)})} \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2^2}{2h^2}\right)}.$$
    $t \leftarrow t + 1$.
until $\|\boldsymbol{x}^{(t+1)} - \boldsymbol{x}^{(t)}\|_2$ is below a given threshold (convergence).
```

算法 1 正交匹配追踪 (OMP)

```
初始化残差: $\boldsymbol{r} = \boldsymbol{y}$
对 $t = 1$ 至 $K$ 做
    找到相关性最高的特征: $j = \arg\max_j |r^T \boldsymbol{x}_j|$, 其中 $\boldsymbol{x}_j$ 是第 j 行的 $\boldsymbol{X}$ (第 j 个 特征)。
    计算 权重: $\boldsymbol{w}_j = \arg\min_{w_j} \|\boldsymbol{r} - \boldsymbol{x}_j \boldsymbol{w}_j\|^2$
    更新残差: $\boldsymbol{r} = \boldsymbol{r} - \boldsymbol{x}_j \boldsymbol{w}_j$
结束循环
```

算法 2 RANSAC (随机抽样一致性)

```
已给出: 训练 集合 $D = \{x_i, y_i\}$, 门槛 $\epsilon$, 损失函数 $L = \sum l(x, y)$
通过计算预测误差将所有数据分类为内点或异常值 $l$ 且与阈值比较 $\epsilon$. (通常设置为 median 绝对偏差 (MAD) 的 $y$, i.e. $\mathrm{median}(|y_i - \mathrm{median}(y)|)$)
内点的集合被称为 一致集。
保存具有最多内点的模型。
使用最大的一致集来学习最终模型。
```

算法 3 k-均值 聚类

```
输入: 数据点 $\{\boldsymbol{x}_i\}_{i=1}^n \subset \mathbb{R}^d$, 数量 簇 $k$.
初始化: 选择初始 簇 个中心 $\{\boldsymbol{c}_j^{(0)}\}_{j=1}^k \subset \mathbb{R}^d$.
设置迭代计数器 $t \leftarrow 0$.
重复
    分配 步骤: 对于每个 $i = 1, \ldots, n$, 更新 簇 标签 $z_i^{(t)} \leftarrow \arg\min_{j \in \{1,\ldots,k\}} \|\boldsymbol{x}_i - \boldsymbol{c}_j^{(t)}\|_2^2$.
    更新步骤: 对于每一个 $j = 1, \ldots, k$, 更新 簇 中心 $\boldsymbol{c}_j^{(t+1)} \leftarrow \frac{1}{|\mathcal{C}_j^{(t)}|} \sum_{i \in \mathcal{C}_j^{(t)}} \boldsymbol{x}_i, \quad \mathcal{C}_j^{(t)} := \{\, i \in \{1, \ldots, n\} : z_i^{(t)} = j \,\}$.
    $t \leftarrow t + 1$.
直到收敛。
```

算法 4 EM 高斯混合模型的算法 (GMM)

```
输入: 数据点 $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^d$, 数量 簇 $K$, 初始参数用于均值 $\{\boldsymbol{\mu}_j\}_{j=1}^K$, 协方差 $\{\Sigma_j\}_{j=1}^K$, 和 权重 $\{\pi_j\}_{j=1}^K$.
设置迭代计数器 $t \leftarrow 0$.
重复
    E步骤: 计算每个点的簇隶属度 $\boldsymbol{x}_i$: $\hat{z}_{ij} = p(z_i = j|\boldsymbol{x}_i) = \frac{\pi_j \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_j, \Sigma_j)}{\sum_{k=1}^K \pi_k \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_k, \Sigma_k)}$.
    M步骤: 更新每个高斯 簇的参数: 点的软计数在 簇 $j$: $N_j = \sum_{i=1}^N \hat{z}_{ij}$.
    $\pi_j \leftarrow \frac{N_j}{N}, \boldsymbol{\mu}_j \leftarrow \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} \boldsymbol{x}_i, \Sigma_j \leftarrow \frac{1}{N_j} \sum_{i=1}^N \hat{z}_{ij} (\boldsymbol{x}_i - \boldsymbol{\mu}_j)(\boldsymbol{x}_i - \boldsymbol{\mu}_j)^\top$.
    $t \leftarrow t + 1$.
直到收敛。
```

算法 1 均值-移位算法

```
输入: 数据点 $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^d$, 带宽 参数 $h$, 起点 $\boldsymbol{x}^{(0)}$.
设置迭代计数器 $t \leftarrow 0$.
重复
    1) 查找……的邻居 $\boldsymbol{x}^{(t)}$: 对于每个数据点 $\boldsymbol{x}_i$, 计算距离至 $\boldsymbol{x}^{(t)}$. 找到半径为 带宽 的邻居集合 $h$: $\mathcal{N}(\boldsymbol{x}^{(t)}) = \left\{ \boldsymbol{x}_i : \|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2 \le h \right\}$.
    2) 将点移向邻居的均值: 更新 $\boldsymbol{x}^{(t+1)}$ 到权重均值的点在 $\mathcal{N}(\boldsymbol{x}^{(t)})$ 使用 高斯 核函数:
        $$\boldsymbol{x}^{(t+1)} = \frac{\sum_{\boldsymbol{x}_i \in \mathcal{N}(\boldsymbol{x}^{(t)})} \boldsymbol{x}_i \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2^2}{2h^2}\right)}{\sum_{\boldsymbol{x}_i \in \mathcal{N}(\boldsymbol{x}^{(t)})} \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}^{(t)}\|_2^2}{2h^2}\right)}.$$
    $t \leftarrow t + 1$.
直到 $\|\boldsymbol{x}^{(t+1)} - \boldsymbol{x}^{(t)}\|_2$ 低于给定阈值（收敛）。
```